[LOGO ÉCOLE - À GAUCHE]

[LOGO MARKASOLUTION - À DROITE]

RAPPORT DE STAGE D'INGÉNIERIE

Conception et développement d'une application web FleetWise pour l'optimisation des coûts logistiques dans le secteur du transport

Présenté par : [Nom et Prénom de l'étudiant]
N° d'étudiant : [Numéro d'étudiant]

Filière : Génie Informatique
Spécialité : Ingénierie Logicielle
Année universitaire : 2024-2025

Encadrant académique : [Nom], [Grade]
[Établissement]

Encadrant en entreprise : Safae MAZOZI
Ingénieur Développeur Principal
MarkaSolution

Entreprise d'accueil

MarkaSolution (SARL)
Casablanca, Maroc

Période du stage : [Dates exactes - 5 semaines]

Lieu et date de dépôt : Casablanca, [Date]

REMERCIEMENTS

Je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué à la réussite de ce stage d'initiation et à la réalisation de ce projet.

Mes premiers remerciements s'adressent à Madame Safae MAZOZI, Ingénieur Développeur Principal chez MarkaSolution, qui a accepté de m'encadrer durant ces cinq semaines. Sa confiance, sa disponibilité et ses conseils avisés ont été déterminants dans la réussite de ce projet. Ses retours constructifs et son expertise technique m'ont permis d'évoluer rapidement et de mener à bien le développement de l'application FleetWise.

Je remercie également l'ensemble de l'équipe de MarkaSolution pour leur accueil chaleureux et l'environnement de travail stimulant qu'ils ont su créer. Cette expérience m'a permis de découvrir concrètement le monde de l'entreprise et les réalités du métier d'ingénieur en développement logiciel.

Mes remerciements vont aussi à mon encadrant académique [Nom] pour son suivi et ses orientations qui ont enrichi cette expérience professionnelle.

Enfin, je remercie ma famille et mes proches pour leur soutien constant tout au long de ma formation.

RÉSUMÉ

Ce rapport présente le travail réalisé durant un stage d'initiation de 5 semaines au sein de MarkaSolution, une Entreprise de Services du Numérique marocaine basée à Casablanca. Le projet consistait à concevoir et développer une application web baptisée "FleetWise" pour optimiser la gestion des coûts logistiques d'une société de transport marocaine.

La problématique adressée concernait l'absence d'un système d'information centralisé pour le calcul des coûts de transport, entraînant des processus manuels lents et sujets à erreurs. L'objectif était de créer un outil d'aide à la décision capable de calculer automatiquement les itinéraires optimisés pour poids lourds et d'évaluer précisément les coûts (carburant, chauffeur, péages).

La méthodologie adoptée a suivi le cycle complet de développement logiciel : analyse des besoins, conception (MCD/MLD, UML), réalisation technique et validation. L'architecture retenue est une Single Page Application avec un backend Django/DRF et un frontend React, intégrant l'API OpenRouteService pour les données géographiques.

Les résultats obtenus démontrent un gain significatif en termes de précision des calculs, de rapidité (temps de réponse < 5 secondes) et d'optimisation des processus métier. L'application livre une solution complète incluant un tableau de bord opérationnel, un module d'assignation de missions et un panneau d'administration pour la gestion des données.

Mots-clés : *Optimisation logistique, Application web, Django, React, API REST, Transport*

ABSTRACT

This report presents the work carried out during a 5-week initiation internship at Marka-Solution, a Moroccan Digital Services Company based in Casablanca. The project consisted of designing and developing a web application called “FleetWise” to optimize logistics cost management for a Moroccan transport company.

The problem addressed concerned the absence of a centralized information system for calculating transport costs, resulting in slow and error-prone manual processes. The objective was to create a decision support tool capable of automatically calculating optimized routes for heavy vehicles and accurately evaluating costs (fuel, driver, tolls).

The methodology adopted followed the complete software development cycle : needs analysis, design (MCD/MLD, UML), technical implementation and validation. The chosen architecture is a Single Page Application with a Django/DRF backend and a React frontend, integrating the OpenRouteService API for geographical data.

The results obtained demonstrate a significant gain in terms of calculation accuracy, speed (response time < 5 seconds) and business process optimization. The application delivers a complete solution including an operational dashboard, a mission assignment module and an administration panel for data management.

Keywords: *Logistics optimization, Web application, Django, React, REST API, Transport*

TABLE DES MATIÈRES

Pages préliminaires

Remerciements i
Résumé ii
Abstract iii
Table des matières iv
Liste des figures vi
Liste des tableaux vii
Liste des abréviations viii

Introduction générale 1

Chapitre 1 : Présentation de l'Entreprise 3

1.1 Fiche d'identité 3
1.2 Domaines d'expertise 4
1.3 Moyens matériels et techniques 5
1.4 Rôle du stagiaire et périmètre de la mission 6

Chapitre 2 : Contexte, Problématique et Objectifs 8

2.1 Contexte métier 8
2.2 Problématique précise 10
2.3 Objectifs du projet 11
2.3.1 Objectifs fonctionnels 11
2.3.2 Objectifs non fonctionnels 12

Chapitre 3 : Analyse & Spécifications 14

3.1 Recueil et analyse des besoins 14
3.2 Outils technologiques et environnement de travail 16
3.2.1 Technologies et frameworks 17
3.2.2 Environnement de travail et outils 20
3.2.3 Services et protocoles externes 21
3.3 Spécifications fonctionnelles détaillées 22
3.4 Conception logicielle 24
3.4.1 Architecture générale 24
3.4.2 Conception de la base de données (MCD/MLD) 25
3.4.3 Diagrammes UML 26
3.5 Spécifications API / échanges 29
3.6 Contraintes & risques 30

Chapitre 4 : Réalisation Technique 32

4.1 Environnement de développement 32
4.2 Backend (description détaillée) 33
4.2.1 Structure du projet et des modèles 33
4.2.2 Services métier : Le CalculateurDeCout 34
4.2.3 Tests unitaires backend 35
4.3 Frontend (description détaillée) 37
4.3.1 Architecture des composants 37
4.3.2 Gestion d'état et appels API 38
4.4 Présentation visuelle de l'application 39
4.4.1 Le tableau de bord : le cœur opérationnel 39
4.4.2 Le panneau d'administration : contrôle et gestion des données 43

Chapitre 5 : Tests, Validation et Résultats 49

5.1 Stratégie de tests 49
5.2 Cas d'études / Scénarios réels 51
5.3 Résultats mesurés 53
5.4 Retour utilisateur et validation 54
5.5 Limites identifiées 55

Chapitre 6 : Analyse Critique, Enseignements et Perspectives 56

6.1 Difficultés rencontrées 56
6.2 Solutions et compromis adoptés 58
6.3 Apports du stage (personnels et techniques) 59

Conclusion générale 61

Bibliographie / Webographie 63

LISTE DES FIGURES

Figure 1.1 : Vue d'ensemble des locaux de MarkaSolution 3
Figure 1.2 : Cycle de développement du projet FleetWise 7
Figure 2.1 : Workflow opérationnel de gestion des missions de transport 9
Figure 2.2 : Enter Caption 11
Figure 3.1 : Logo de Python et Django 17
Figure 3.2 : Logo de React/Js 18
Figure 3.3 : Logo de MySQL 19
Figure 3.4 : Logo de VS Code 21
Figure 3.5 : Logo de OpenRouteService API Platform 21
Figure 3.6 : Diagramme des Cas d'Utilisation de FleetWise 22
Figure 3.7 : Modèle Conceptuel de Données (MCD) de FleetWise 25
Figure 3.8 : Modèle Logique de Données (MLD) de FleetWise 25
Figure 3.9 : Diagramme de Séquence du calcul d'itinéraire 26
Figure 3.10 : Diagramme de Séquence de l'assignation de mission par email 27
Figure 3.11 : Diagramme de Classes simplifié 28
Figure 4.1 : Page d'accueil de FleetWise 40
Figure 4.2 : Système de suggestion de villes pour une saisie assistée 40
Figure 4.3 : Affichage structuré des résultats d'analyse après calcul 41
Figure 4.4 : Fenêtre modale pour l'assignation de la mission au chauffeur 42
Figure 4.5 : Interface de gestion des paramètres économiques du système 44
Figure 4.6 : Tableau de gestion de la flotte de camions 45
Figure 4.7 : Fenêtre modale pour la modification des informations d'un véhicule 46
Figure 4.8 : Interface de gestion des chauffeurs 47
Figure 4.9 : Interface de gestion des tarifs de péage 48
Figure 5.1 : Capture d'écran annotée des résultats pour le trajet Casablanca → Agadir 52

LISTE DES TABLEAUX

Tableau 1.1 : Fiche d'Identité de MarkaSolution 4
Tableau 2.1 : Objectifs non fonctionnels du projet FleetWise 12
Tableau 3.1 : Tableau Récapitulatif des Besoins Fonctionnels et Non-Fonctionnels 15
Tableau 3.2 : Spécifications des Endpoints Principaux de l'API FleetWise 29
Tableau 4.1 : Principaux scénarios de test pour le CalculateurDeCout 36
Tableau 5.1 : Analyse des Résultats pour le Trajet Recommandé Casablanca → Agadir 53

LISTE DES ABRÉVIATIONS

API	Application Programming Interface
CSRF	Cross-Site Request Forgery
CRUD	Create, Read, Update, Delete
DRF	Django REST Framework
ESN	Entreprise de Services du Numérique
HGV	Heavy Goods Vehicle
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
MAD	Dirham Marocain
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
MVT	Modèle-Vue-Template
ORM	Object-Relational Mapping
ORS	OpenRouteService
PME	Petites et Moyennes Entreprises
REST	Representational State Transfer
SARL	Société À Responsabilité Limitée
SGBDR	Système de Gestion de Base de Données Relationnelle
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
UML	Unified Modeling Language
VS Code	Visual Studio Code

INTRODUCTION GÉNÉRALE

Le secteur du transport et de la logistique au Maroc connaît une croissance soutenue, portée par le développement des infrastructures nationales et l'intensification des échanges commerciaux. Cette dynamique s'accompagne d'une digitalisation progressive des processus métier, les entreprises cherchant à optimiser leurs opérations pour rester compétitives dans un marché en constante évolution. Dans ce contexte, les outils d'aide à la décision et d'optimisation des coûts deviennent des leviers stratégiques essentiels pour les acteurs du secteur.

MarkaSolution, Entreprise de Services du Numérique (ESN) marocaine fondée en 2015 et basée à Casablanca, s'est positionnée comme un partenaire privilégié pour accompagner cette transformation digitale. Spécialisée dans la conception et le développement de solutions logicielles sur mesure, elle compte aujourd'hui 72 employés et intervient auprès d'une clientèle diversifiée à travers des projets alliant expertise technique et compréhension des enjeux métier.

C'est dans le cadre d'une collaboration avec une société de transport marocaine qu'a émergé la problématique centrale de ce stage : l'absence d'un système d'information centralisé pour la gestion et l'optimisation des coûts logistiques. Cette entreprise, représentative du tissu économique national, gérait ses missions de transport par des processus entièrement manuels, s'appuyant sur l'expérience du directeur d'exploitation pour estimer les coûts de carburant, de main-d'œuvre et de péages. Cette approche, bien que fonctionnelle, présentait des limites critiques : lenteur des calculs, risques d'erreurs humaines, manque de traçabilité et absence d'optimisation des itinéraires.

L'objectif principal de ce stage d'initiation était de concevoir et développer une application web complète, baptisée "FleetWise", capable de répondre à ces défis opérationnels. Sur le plan pédagogique, cette mission offrait l'opportunité de mettre en pratique les connaissances théoriques acquises durant ma formation d'ingénieur, en particulier dans les domaines du développement web, de la conception de bases de données et de l'intégration de services externes. Sur le plan professionnel, elle permettait de découvrir les réalités du métier d'ingénieur en entreprise et de contribuer concrètement à la résolution d'une problématique métier réelle.

La méthodologie adoptée s'est inspirée des bonnes pratiques de l'ingénierie logicielle, en suivant un cycle de développement structuré : phase d'analyse et de recueil des besoins, conception

architecturale et technique, implémentation avec des technologies modernes (Django, React, MySQL), puis validation par des tests fonctionnels et techniques. L'approche de développement a été itérative, avec des points de validation réguliers auprès de l'encadrant professionnel pour s'assurer de l'adéquation entre les fonctionnalités développées et les attentes métier.

> Ce rapport s'articule autour de six chapitres principaux. Le premier chapitre présente le cadre professionnel de la mission, détaillant l'entreprise d'accueil et le périmètre du stage. Le deuxième chapitre expose le contexte métier, la problématique identifiée et les objectifs fixés pour le projet. Le troisième chapitre détaille la phase d'analyse et de spécifications, incluant la conception architecturale et technique de la solution. Le quatrième chapitre décrit la réalisation technique, couvrant à la fois le développement backend et frontend. Le cinquième chapitre présente la stratégie de validation adoptée et les résultats obtenus. Enfin, le sixième chapitre propose une analyse critique du travail réalisé, des enseignements tirés et des perspectives d'évolution du projet.

1 Présentation de l'Entreprise

Ce premier chapitre a pour objectif de présenter le cadre professionnel au sein duquel j'ai eu l'opportunité d'effectuer mon stage d'initiation. Il me semble essentiel de contextualiser la mission qui m'a été confiée en décrivant l'organisme d'accueil, MarkaSolution, son positionnement sur le marché, son organisation interne ainsi que les moyens techniques qui ont été mis à ma disposition. Cette présentation permettra, je l'espère, de mieux appréhender l'environnement dans lequel j'ai conçu et développé le projet "FleetWise".

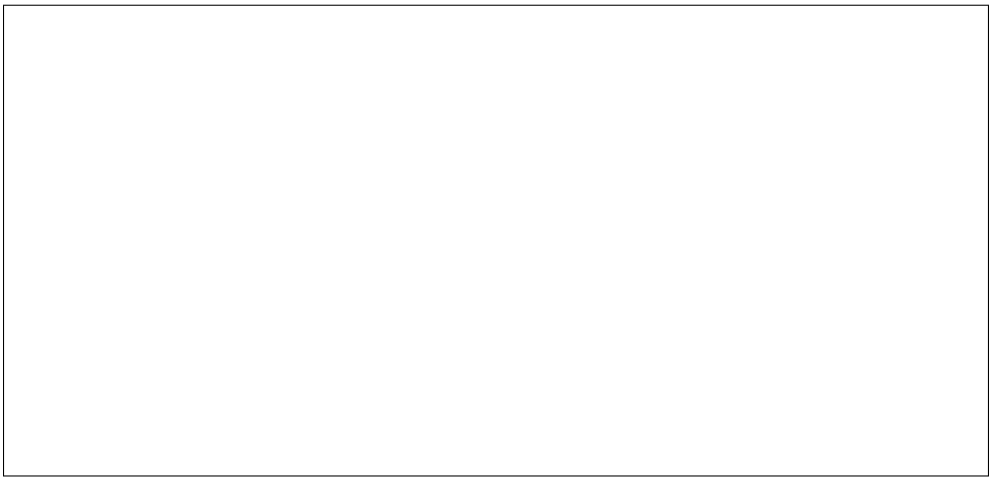


FIGURE 1.1. Vue d'ensemble des locaux de MarkaSolution

1.1 Fiche d'identité

Mon stage s'est déroulé au sein de MarkaSolution, une Entreprise de Services du Numérique (ESN) marocaine que j'ai trouvée particulièrement dynamique. Spécialisée dans la conception et le développement de solutions digitales sur mesure, elle est ancrée dans l'écosystème technologique de Casablanca. J'ai pu observer durant mon stage qu'elle se positionne comme un partenaire stratégique pour les sociétés cherchant à opérer leur transformation numérique.

TABLE 1.1. Fiche d'Identité de MarkaSolution

Raison Sociale	MarkaSolution
Forme Juridique	SARL
Année de Création	2015
Secteur d'Activité	Ingénierie Informatique
Siège Social	Maroc
Effectif	Nombre d'employés : 72

Au-delà de cette fiche signalétique, j'ai pu constater que MarkaSolution se distingue par une approche polyvalente, couvrant un large spectre de services technologiques. Cette polyvalence lui permet de proposer des solutions intégrées qui répondent de manière globale aux défis de ses clients, de la conception logicielle initiale au support post-déploiement.

1.2 Domaines d'expertise

L'expertise de MarkaSolution, telle que je l'ai perçue, s'articule autour de plusieurs pôles de compétences complémentaires. Cette organisation lui permet d'offrir une chaîne de valeur complète dans le domaine du numérique. Les activités principales que j'ai pu identifier sont :

- **Développement de sites web et d'applications sur mesure :** C'est le cœur de métier de l'entreprise. J'ai vu les équipes travailler sur la réalisation de plateformes web complexes, d'applications métier et de solutions e-commerce, en s'appuyant sur des architectures modernes et des technologies éprouvées.
- **Conseil technique et stratégique :** J'ai compris que MarkaSolution ne se contente pas de développer, mais accompagne également ses clients dans la définition de leur stratégie digitale, l'audit de leurs systèmes d'information et le choix des architectures techniques les plus adaptées à leurs enjeux.
- **Conception de logiciels :** L'entreprise possède une forte expertise dans le cycle de vie complet du développement logiciel. J'ai pu voir l'application de méthodes de conception structurées, de l'analyse des besoins à l'implémentation et la maintenance des solutions.
- **Support et maintenance des solutions numériques :** J'ai aussi noté l'importance accordée à la continuité des opérations pour les clients. MarkaSolution propose des contrats de support pour garantir la disponibilité, la sécurité et l'évolutivité des plateformes développées.

C'est dans le cadre de son expertise en solutions logicielles pour le secteur de la logistique que le projet qui m'a été confié a pris tout son sens.

1.3 Moyens matériels et techniques

Pour mener à bien ma mission, j'ai été intégré dans un environnement technique moderne et structuré. L'entreprise met à la disposition de ses équipes des moyens matériels et logiciels qui favorisent l'efficacité et la collaboration.

L'environnement de développement standard repose sur une pile technologique cohérente que j'ai eu l'occasion d'utiliser :

- **Langages et Frameworks :** Python avec le framework Django pour le backend ; JavaScript avec la bibliothèque React pour le frontend.
- **Base de Données :** L'environnement de développement privilégie SQLite pour sa légèreté, avec une cible de production sur MySQL pour sa robustesse.
- **Outils Collaboratifs :** L'utilisation de **Git** pour le contrôle de version est systématique, avec des dépôts centralisés sur [GitHub/GitLab, à compléter]. J'ai également utilisé des outils de communication comme [Slack/Teams, à compléter] qui fluidifiaient les échanges au sein de l'équipe.
- **Infrastructure :** Bien que mon projet ait été développé localement, j'ai pu observer que l'entreprise s'appuie sur des solutions cloud comme [AWS/Azure, à compléter] pour le déploiement des projets en production, ce qui m'a donné un aperçu des pratiques DevOps actuelles.

Ces moyens m'ont permis de travailler dans des conditions optimales, très proches de celles d'un ingénieur en poste.

1.4 Rôle du stagiaire et périmètre de la mission

Mon intégration au sein de l'équipe technique travaillant sur le projet pour la société de transport a été immédiate et concrète. Mon stage a rapidement dépassé le cadre d'une simple mission d'observation pour devenir une véritable expérience de développement, riche en responsabilités.

Rattaché hiérarchiquement à **Safae Mazozi, Ingénieur Développeur Principal**, ma mission s'est articulée autour de la **conception et du développement de l'application web "FleetWise"**. Ce projet, que j'ai mené en autonomie supervisée, avait pour objectif de créer un outil d'aide à la décision pour optimiser la gestion des coûts de transport.

Le périmètre de ma mission couvrait l'intégralité du cycle de vie du projet :

1. **Phase d'analyse :** J'ai commencé par une étude du besoin, ce qui m'a conduit à définir le cahier des charges fonctionnel et non-fonctionnel de l'application.
2. **Phase de conception :** J'ai ensuite modélisé l'architecture de la solution, conçu la base de données en utilisant les approches MCD/MLD et formalisé les interactions via des

diagrammes UML.

3. **Phase de réalisation :** Le cœur de ma mission a été le développement full-stack de l'application, en utilisant Django pour le backend et React pour le frontend.
4. **Phase de validation :** Enfin, j'ai effectué une série de tests fonctionnels pour m'assurer de la fiabilité et de la précision des calculs de la solution développée.

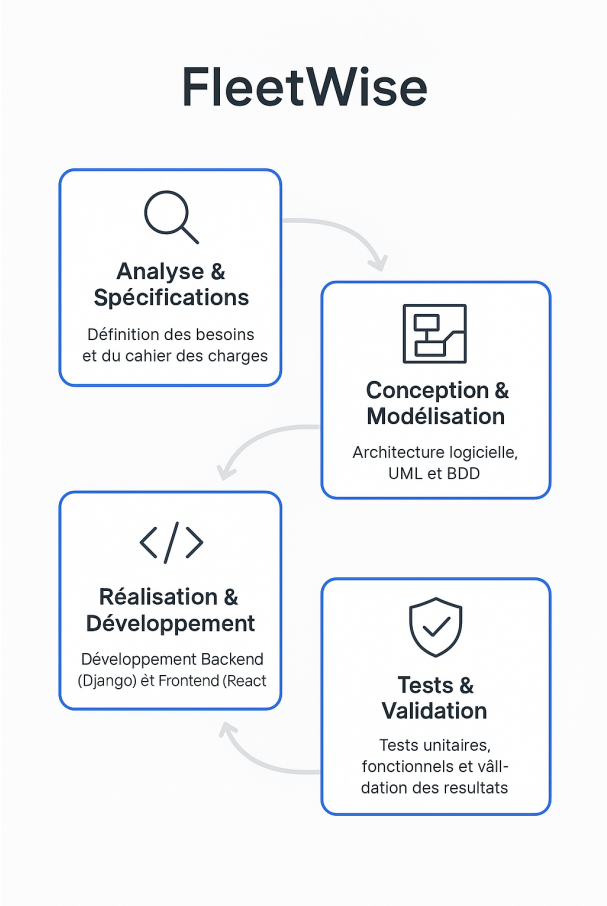


FIGURE 1.2. Cycle de développement du projet FleetWise

Les livrables que je devais produire à l'issue de cette mission étaient l'application web fonctionnelle, son code source documenté, et le présent rapport de stage. Cette mission a ainsi constitué une opportunité exceptionnelle de mettre en pratique les connaissances théoriques acquises durant ma formation d'ingénieur dans un contexte professionnel exigeant et très formateur.

2

Contexte, Problématique et Objectifs

Ce deuxième chapitre se concentre sur le cœur de ma mission. L'objectif ici est de poser les fondations du projet "FleetWise" en décrivant en détail le contexte métier de l'entreprise cliente, une société de transport marocaine. J'analyserai le workflow opérationnel existant pour en extraire une problématique précise et mesurable. Finalement, je traduirai cette problématique en une série d'objectifs fonctionnels et non-fonctionnels qui ont constitué le cahier des charges et la feuille de route de mon travail de développement.

2.1 Contexte métier

Le projet qui m'a été confié s'inscrit dans le cadre d'une collaboration entre MarkaSolution et une société de transport et de travaux divers entreprise kompanio sarl basée à Rabat. Cette entreprise, une PME typique du tissu économique marocain, gère une flotte hétérogène de véhicules allant de l'utilitaire léger aux camions poids lourds. Sa principale activité consiste à répondre à des demandes de transport de marchandises pour une clientèle B2B à travers tout le royaume.

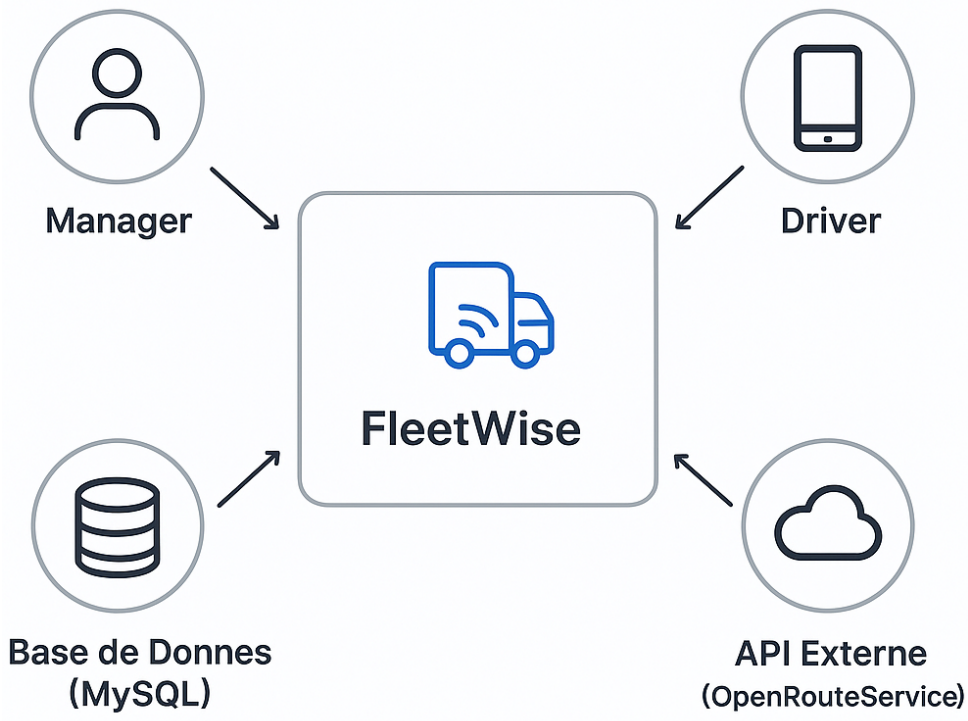


FIGURE 2.1. Workflow opérationnel de gestion des missions de transport

Lors de ma phase d'analyse initiale, j'ai pu observer et documenter le processus de gestion des missions, de la réception de l'appel d'offres à l'assignation au chauffeur. J'ai constaté que ce workflow, bien que fonctionnel, reposait entièrement sur des processus manuels et l'expérience du directeur d'exploitation. Le déroulement type d'une demande était le suivant :

1. **Réception de la demande :** Un client contacte l'entreprise par téléphone pour un besoin de transport entre une ville A et une ville B.
2. **Estimation et Calcul :** Le directeur, pour fournir un devis, entame un processus de calcul mental et manuel. Il estime la distance en se basant sur sa connaissance des routes, consulte le prix du carburant du jour, et tente d'estimer la durée du trajet. Le coût des péages, s'il y en a, est souvent une approximation ou est vérifié ponctuellement sur des sources externes.
3. **Cotation :** À l'aide d'une calculatrice, il additionne ces coûts estimés et y applique une marge bénéficiaire qu'il juge adéquate, avant de communiquer le prix final au client.
4. **Assignation :** Une fois le devis accepté, la mission est assignée oralement ou par message informel à un chauffeur jugé disponible.

Ce processus traditionnel présentait plusieurs limites critiques qui freinaient l'efficacité et la croissance de l'entreprise. J'ai pu identifier un manque de traçabilité, une forte dépendance à l'expertise d'une seule personne, et une absence totale d'optimisation.

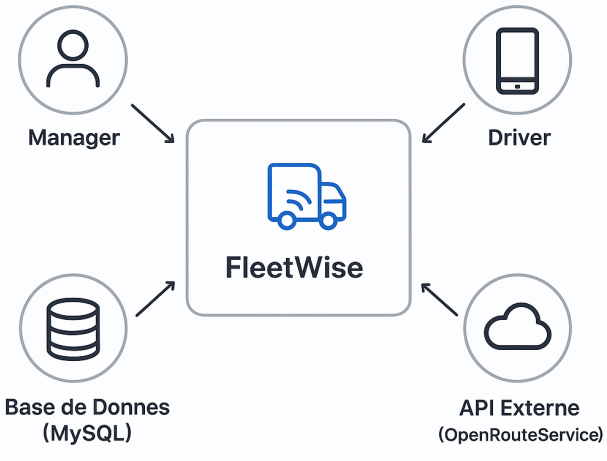


FIGURE 2.2. Enter Caption

2.2 Problématique précise

L'analyse du workflow existant a mis en lumière une problématique centrale :

L'absence d'un système d'information centralisé et automatisé pour la gestion des coûts logistiques, entraînant un manque de précision, une perte de temps significative et un risque financier pour l'entreprise.

Cette problématique principale peut être décomposée en plusieurs points de friction mesurables :

— **Absence d'automatisation dans le calcul des coûts :** Les calculs manuels sont non seulement lents, mais ils sont aussi sujets à des erreurs humaines qui peuvent impacter négativement la marge bénéficiaire de chaque mission.
— **Manque d'optimisation des itinéraires :** Le choix de l'itinéraire se basait sur l'habitude plutôt que sur une analyse en temps réel des alternatives. Il n'existait aucun moyen de comparer rapidement l'itinéraire le plus rapide face au plus économique.
— **Difficultés d'assignation et de suivi :** L'assignation orale des missions manquait de formalisme et de traçabilité, rendant le suivi des opérations complexe.
— **Dépendance à des données non centralisées :** Les informations vitales (consommation des véhicules, tarifs des péages, coût horaire des chauffeurs) étaient soit stockées de manière informelle, soit non documentées, empêchant toute standardisation.

Il est donc apparu impératif de concevoir une solution digitale pour répondre à ces enjeux et doter l'entreprise d'un avantage concurrentiel.

2.3 Objectifs du projet

Pour répondre à cette problématique, j'ai défini une série d'objectifs clairs et mesurables pour le projet "FleetWise".

2.3.1 Objectifs fonctionnels

Les fonctionnalités que l'application devait impérativement offrir sont les suivantes :

- **Calcul automatisé et multi-options d'itinéraires :** Intégrer l'API **OpenRouteService** pour calculer non pas un, mais plusieurs itinéraires possibles entre deux villes, en utilisant le profil `driving-hgv` pour une pertinence maximale pour les poids lourds.
- **Évaluation complète et transparente des coûts :** Pour chaque itinéraire, le système doit calculer et détailler les trois composantes majeures du coût : le **coût du carburant** (basé sur la distance précise et la consommation du véhicule), le **coût du chauffeur** (basé sur la durée précise et un tarif horaire), et le **coût des péages**.
- **Présentation d'une recommandation intelligente :** Le système doit analyser les différentes options et présenter un **"Itinéraire Recommandé"** basé sur un équilibre coût/temps, afin de faciliter la prise de décision.
- **Assignation de mission formalisée :** Permettre au gestionnaire, après avoir choisi un itinéraire, de l'assigner à un chauffeur via une interface dédiée, et de déclencher l'**envoi automatique d'un email** contenant les détails opérationnels de la mission.
- **Interface 100% en français :** L'ensemble de l'interface utilisateur doit être en français pour une adoption immédiate par les utilisateurs.

2.3.2 Objectifs non fonctionnels

La qualité de la solution dépendait également du respect d'exigences techniques et qualitatives :

TABLE 2.1. Objectifs non fonctionnels du projet FleetWise

Critère	Exigence
Performance	Le temps total entre la soumission du formulaire et l'affichage des résultats ne doit pas dépasser **10 secondes** pour garantir une expérience utilisateur fluide.
Sécurité	L'application doit être sécurisée via le protocole **HTTPS**, et le framework backend doit fournir une protection native contre les failles courantes comme le **CSRF**.
Fiabilité	Le système doit être robuste, notamment dans sa gestion des appels à l'API externe. Une panne de l'API OpenRouteService ne doit pas entraîner un crash de l'application, mais afficher un message d'erreur clair.
Scalabilité	L'architecture doit permettre d'ajouter facilement de nouveaux véhicules, chauffeurs ou tarifs de péage sans nécessiter de refonte.
Ergonomie	L'interface utilisateur doit être professionnelle, épurée et intuitive, adaptée à un public de managers et de directeurs.

Ces objectifs ont formé le cahier des charges qui a guidé toutes les phases de conception et de développement de l'application "FleetWise".

3 Analyse & Spécifications

Après avoir exposé le contexte métier et les objectifs stratégiques du projet “FleetWise”, ce chapitre se consacre à la phase cruciale de l’ingénierie logicielle : la traduction des besoins en spécifications techniques détaillées. Cette étape est fondamentale car elle constitue le plan directeur qui a guidé l’ensemble du processus de développement. J’y décrirai la méthodologie de recueil des besoins, les spécifications fonctionnelles et non-fonctionnelles qui en ont découlé, l’architecture logicielle retenue, et enfin la conception détaillée de la base de données et des interfaces de programmation (API).

3.1 Recueil et analyse des besoins

Pour garantir que la solution “FleetWise” réponde de manière précise aux problématiques de l’entreprise de transport, j’ai adopté une approche structurée pour le recueil des besoins. Cette démarche s’est appuyée sur des entretiens informels avec mon encadrant professionnel, qui a joué le rôle du “client”, ainsi que sur l’observation du workflow manuel que j’ai pu documenter. Ces échanges m’ont permis de formaliser un cahier des charges initial, que j’ai ensuite affiné tout au long de la phase de conception. Le tableau ci-dessous synthétise les exigences fonctionnelles et non-fonctionnelles qui ont été identifiées et validées.

TABLE 3.1. Tableau Récapitulatif des Besoins Fonctionnels et Non-Fonctionnels

ID	Type	Description du Besoin
BF-01	**Fonctionnel**	**Calcul d'itinéraires optimisés :** Le système doit calculer plusieurs options de trajet entre deux villes en utilisant des données géographiques précises pour les poids lourds.
BF-02	**Fonctionnel**	**Évaluation des coûts :** Le système doit décomposer le coût total en trois postes : carburant (basé sur la consommation du véhicule), chauffeur (basé sur la durée), et péages.
BF-03	**Fonctionnel**	**Gestion des entités (CRUD) :** L'administrateur doit pouvoir gérer la flotte de camions, la liste des chauffeurs, les tarifs de péage, et les paramètres économiques via une interface dédiée.
BF-04	**Fonctionnel**	**Assignation de mission :** Le système doit permettre d'assigner un itinéraire à un chauffeur et de capturer des notes spécifiques à la mission.
BF-05	**Fonctionnel**	**Notification par email :** L'assignation d'une mission doit déclencher l'envoi automatique d'un email au chauffeur concerné.
BNF-01	**Non-Fonctionnel**	**Performance :** Le temps de réponse pour une analyse complète doit être inférieur à 10 secondes.
BNF-02	**Non-Fonctionnel**	**Sécurité :** Les communications doivent être chiffrées (HTTPS), l'application protégée contre les failles CSRF et les données sensibles (clés API) stockées de manière sécurisée.
BNF-03	**Non-Fonctionnel**	**Ergonomie et Langue :** L'interface doit être professionnelle, intuitive, et entièrement en français.
BNF-04	**Non-Fonctionnel**	**Fiabilité :** Le système doit gérer les erreurs provenant des services externes (API ORS) sans interruption de service pour l'utilisateur.

3.2 Outils Technologiques et Environnement de Travail

La réalisation d'un projet d'ingénierie logicielle aussi complet que "FleetWise" repose sur une sélection judicieuse d'outils et de technologies. Chaque brique de notre environnement a été choisie pour sa robustesse, sa modernité et sa capacité à interagir harmonieusement avec les autres. Cette section a pour but de présenter en détail l'écosystème technique que j'ai mis en place pour le développement de l'application.

3.2.1 Technologies et Frameworks

J'ai adopté une architecture moderne découplée, avec une séparation nette entre le backend et le frontend, nécessitant des technologies spécifiques et performantes pour chaque couche.

Python & Django : Pour le développement de toute la logique métier et du serveur backend, mon choix s'est porté sur **Python** en conjonction avec le framework **Django**. Python est reconnu pour sa syntaxe claire et sa vaste collection de bibliothèques. Django, quant à lui, est un framework de haut niveau qui encourage un développement rapide et propre. Sa structure MVT (Modèle-Vue-Template), sa sécurité intégrée et son ORM puissant m'ont permis de construire une base solide et sécurisée pour l'application.



FIGURE 3.1. Logo de Python et Django

Django REST Framework (DRF) : Pour exposer la logique métier du backend au frontend, j'ai utilisé **Django REST Framework**. C'est une boîte à outils puissante et flexible pour la construction d'API Web. DRF a grandement simplifié la création des endpoints REST, la sérialisation des modèles de données en JSON et la gestion de la validation des données entrantes, garantissant ainsi une communication standardisée et fiable avec l'application React.

JavaScript & React : La totalité de l'interface utilisateur a été construite avec **JavaScript** et la bibliothèque **React**. React a été choisi pour son approche déclarative et basée sur les composants, ce qui m'a permis de développer une interface utilisateur complexe, interactive et facile à maintenir. En tant que Single Page Application (SPA), l'expérience utilisateur est extrêmement fluide et rapide, ce qui était un de nos objectifs non-fonctionnels clés.

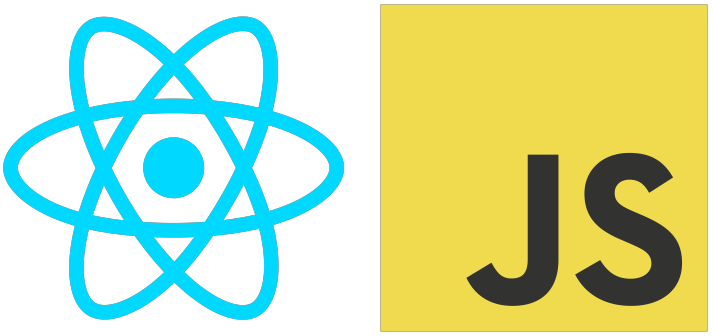


FIGURE 3.2. Logo de React/Js

Base de Données (MySQL) : Pour la gestion des données, j'ai choisi **MySQL** dès le début du projet. En tant que système de gestion de base de données relationnelle (SGBDR) leader de l'industrie, MySQL est réputé pour sa robustesse, ses performances et sa scalabilité. Son utilisation a garanti que l'application reposait sur une fondation de données solide, capable de gérer des accès concurrents et un volume de données croissant, des caractéristiques indispensables pour une application d'entreprise fiable.



FIGURE 3.3. Logo de MySQL

3.2.2 Environnement de Travail et Outils

L'efficacité du développement dépend aussi grandement de la qualité de l'environnement de travail et des outils de support.

Visual Studio Code (VS Code) : Mon principal outil de programmation a été **Visual Studio Code**. Avec sa légèreté, sa polyvalence et son immense écosystème d'extensions, VS Code m'a offert un environnement de développement optimal. J'ai pu bénéficier de fonctionnalités qui ont simplifié mon travail, comme la coloration syntaxique avancée, l'autocomplétion intelligente (IntelliSense), et un terminal intégré. Ses fonctionnalités avancées pour le contrôle de version avec Git et le débogage ont rendu mon code plus performant et ont facilité la collaboration au sein de l'équipe.

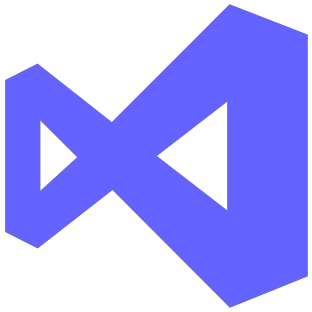


FIGURE 3.4. Logo de VS Code

3.2.3 Services et Protocoles Externes

L'intelligence de "FleetWise" repose en partie sur sa capacité à communiquer avec des services externes via des protocoles standards.

OpenRouteService API : Le cœur du moteur de calcul géographique de "FleetWise" est l'API d'**OpenRouteService**. Ce service a été choisi spécifiquement pour son profil de routage pour poids lourds (`driving-hgv`), une fonctionnalité qui le distingue des autres API grand public et qui était indispensable pour garantir la précision des calculs de distance et de durée pour une flotte de transport.



FIGURE 3.5. Logo de OpenRouteService API Platform

3.3 Spécifications fonctionnelles détaillées

Pour formaliser ces besoins, j'ai utilisé la méthode UML afin de modéliser les interactions entre l'utilisateur et le système. Le diagramme de cas d'utilisation ci-dessous (Figure 3.6) illustre les principales fonctionnalités offertes par "FleetWise" à son utilisateur principal, le gestionnaire.

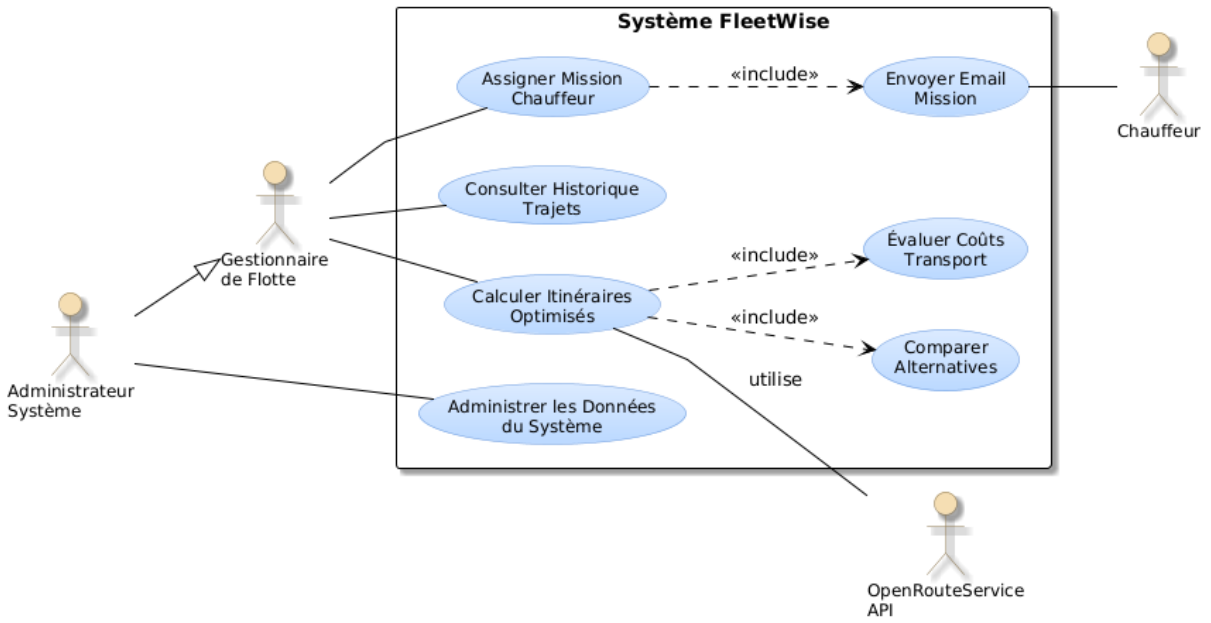


FIGURE 3.6. Diagramme des Cas d'Utilisation de FleetWise

Les cas d'utilisation prioritaires, qui constituent le cœur de l'application, sont le calcul d'un itinéraire et l'assignation d'une mission. Leurs scénarios nominaux sont les suivants :

Scénario "Calculer un itinéraire" :

1. Le gestionnaire saisit la ville de départ et la ville d'arrivée.
2. Il sélectionne un camion dans la liste des véhicules disponibles.
3. Il soumet le formulaire.
4. Le système valide les entrées, interroge l'API OpenRouteService, effectue les calculs de coûts et de profit.
5. Le système affiche une liste d'itinéraires optimisés, avec une option recommandée.

Scénario "Assigner une mission" :

1. Le gestionnaire clique sur le bouton "Assigner" d'un itinéraire recommandé.
2. Le système affiche une fenêtre modale avec les détails du trajet.
3. Le gestionnaire sélectionne un chauffeur dans la liste et ajoute des notes optionnelles.
4. Il confirme l'assignation.
5. Le système envoie une requête à l'API pour déclencher l'envoi de l'email et affiche un message de succès.

3.4 Conception logicielle

L'architecture de l'application a été conçue pour être modulaire, évolutive et maintenable, en séparant clairement les responsabilités entre le client (navigateur) et le serveur.

3.4.1 Architecture générale

J'ai opté pour une architecture de type **Single Page Application (SPA)** avec une **API REST** découplée. Comme l'illustre la Figure **??**, cette architecture se compose de :

1. Un **frontend** développé en **React**, qui gère toute la logique de l'interface utilisateur.
2. Un **backend** développé en **Django**, qui expose une API REST via Django REST Framework. Il contient toute la logique métier, les calculs et les interactions avec la base de données.
3. Une **base de données** relationnelle (**MySQL** en production).
4. Un **service externe** (**OpenRouteService API**) pour les données géographiques.

3.4.2 Conception de la base de données (MCD/MLD)

En m'appuyant sur la méthode Merise, j'ai d'abord conçu le Modèle Conceptuel de Données (MCD) pour représenter les entités et leurs relations. Ce MCD a ensuite été traduit en Modèle Logique de Données (MLD), qui correspond au schéma relationnel de notre base de données. Les principales tables sont : `camion`, `chauffeur`, `trajet_peage` et `parametres_generaux`. La contrainte de ligne unique sur la table `parametres_generaux` a été un point de conception important pour garantir la centralisation des variables économiques.

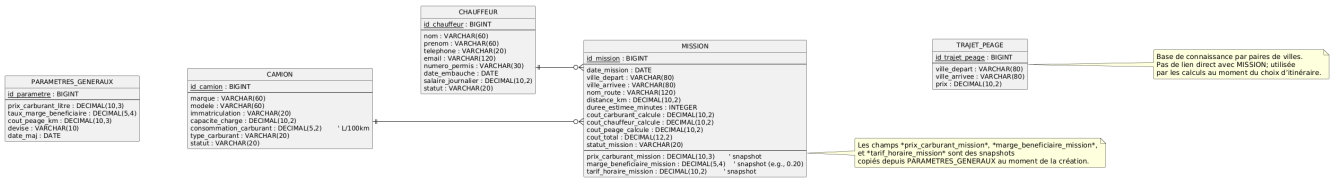


FIGURE 3.7. Modèle Conceptuel de Données (MCD) de FleetWise

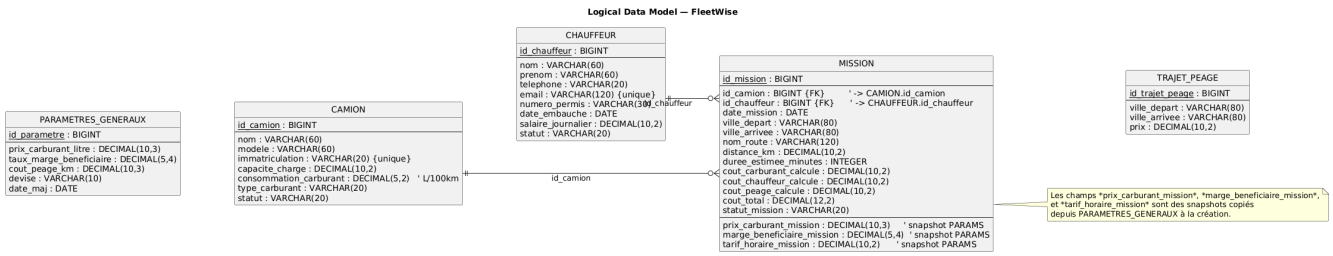


FIGURE 3.8. Modèle Logique de Données (MLD) de FleetWise

3.4.3 Diagrammes UML

Pour détailler le comportement dynamique du système, j'ai modélisé les interactions les plus complexes à l'aide de diagrammes de séquence.

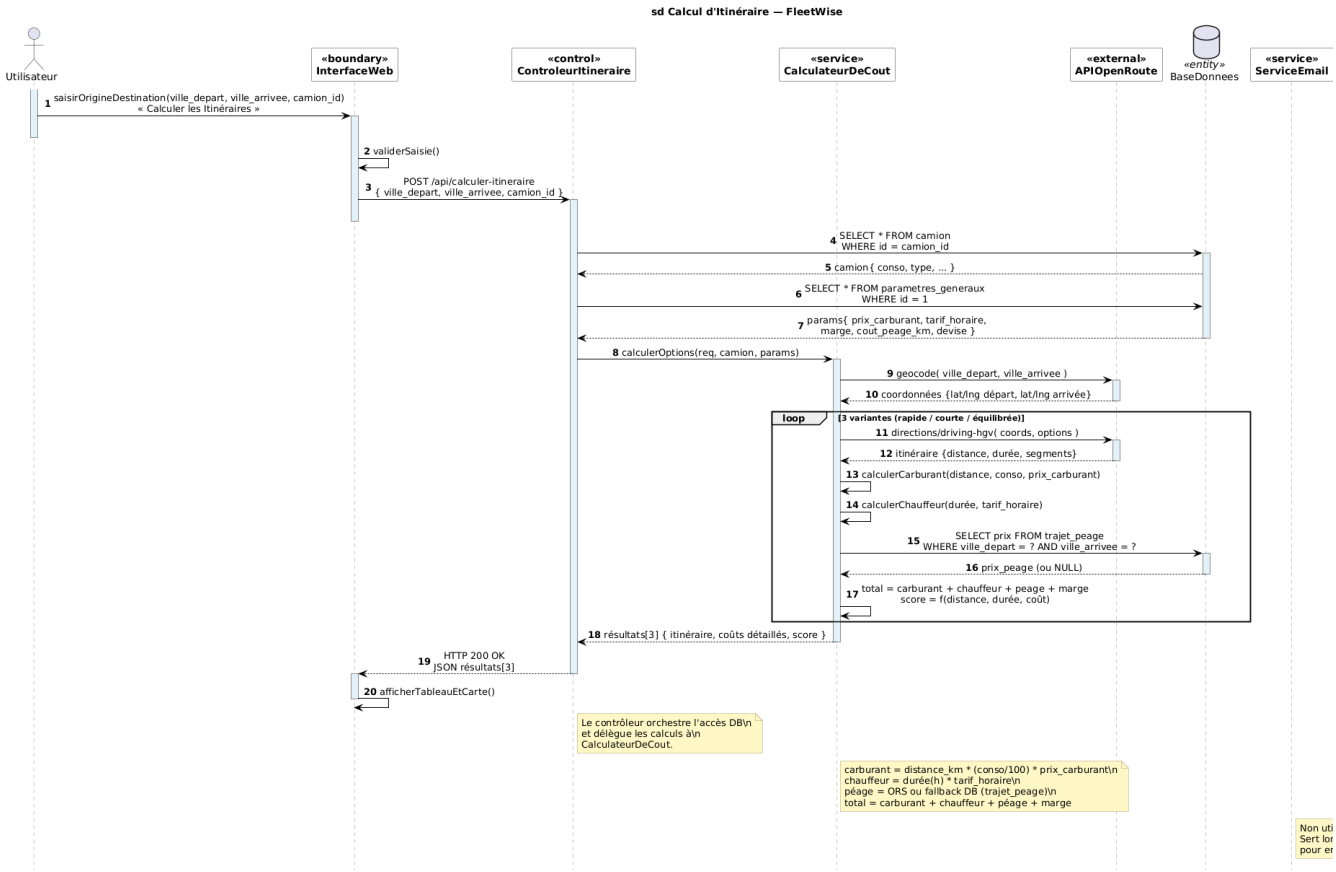


FIGURE 3.9. Diagramme de Séquence du calcul d'itinéraire

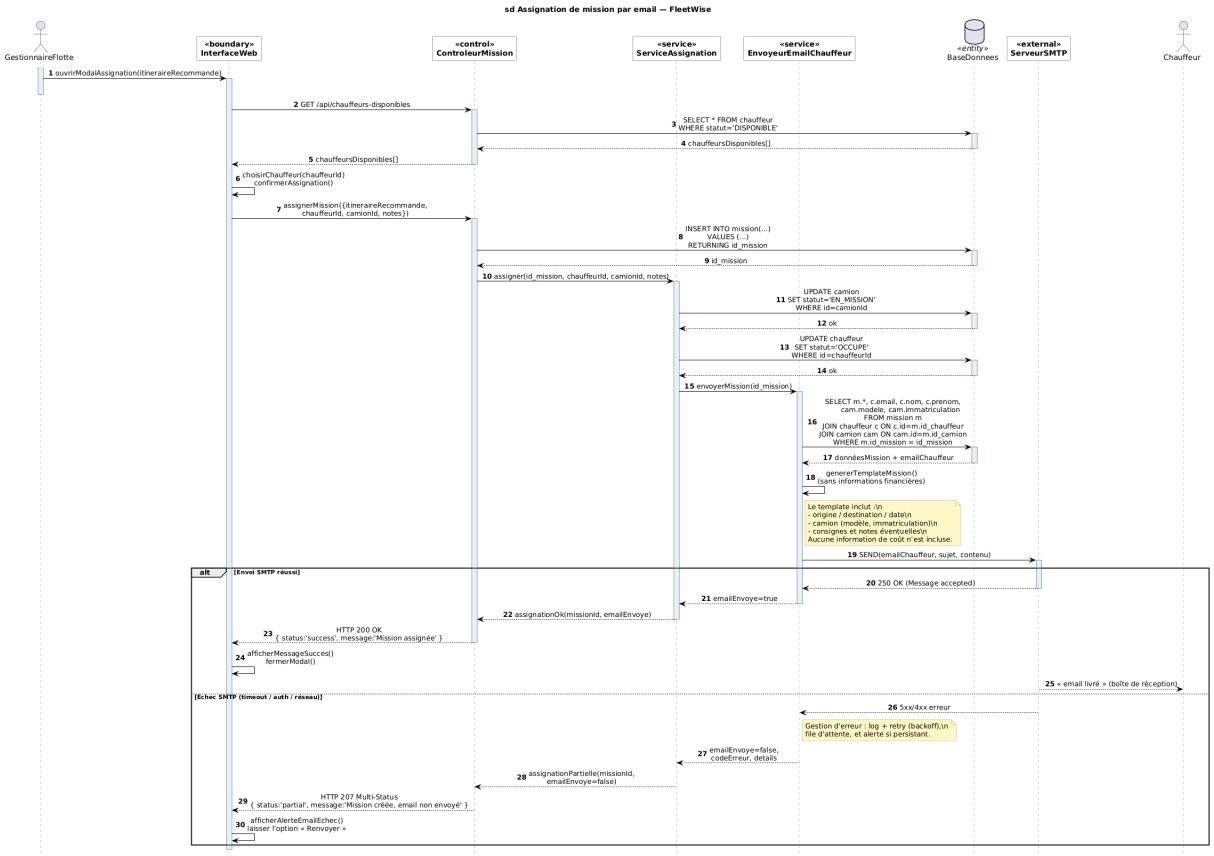


FIGURE 3.10. Diagramme de Séquence de l'assignation de mission par email

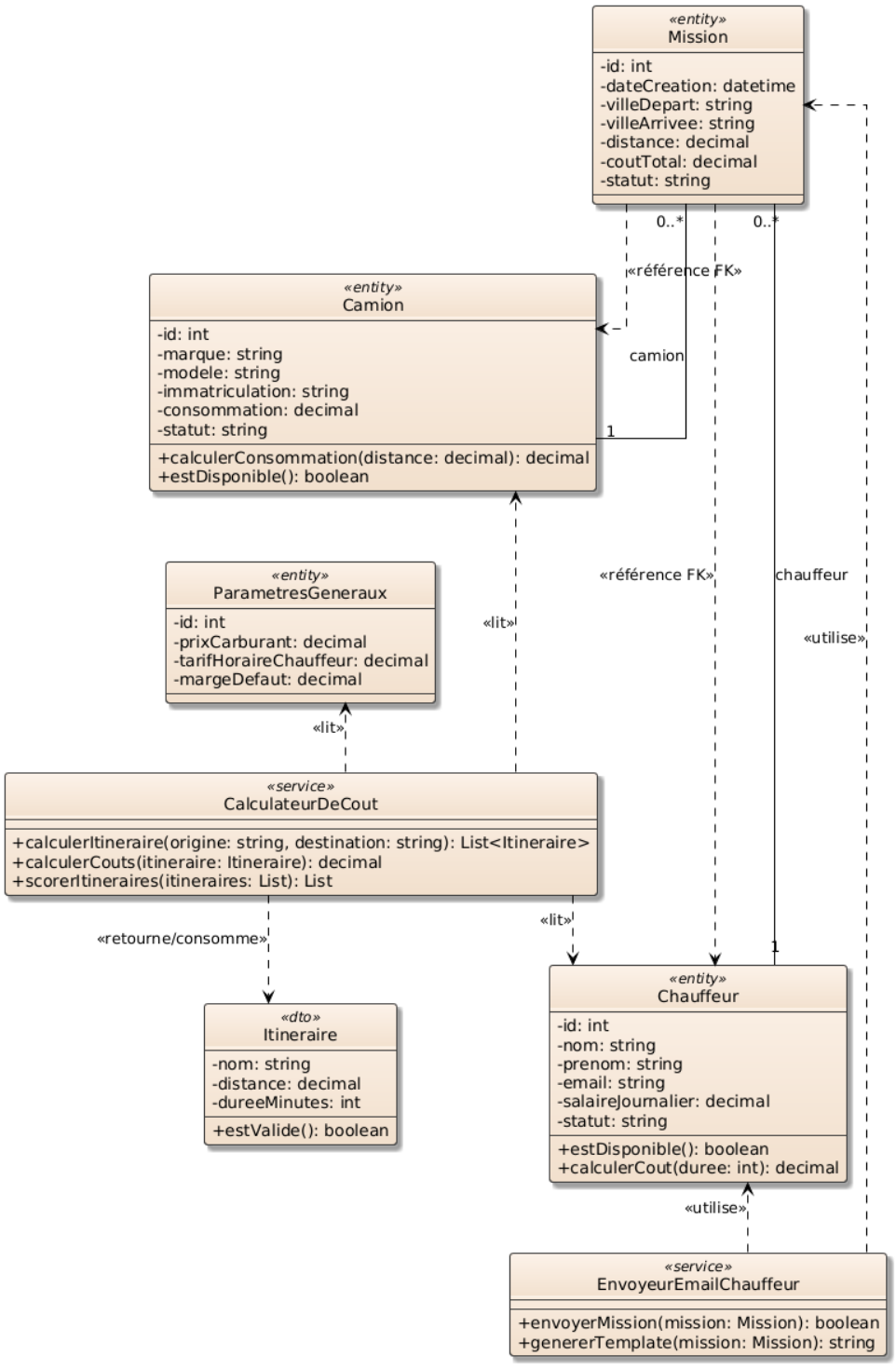


FIGURE 3.11. Diagramme de Classes simplifié

3.5 Spécifications API / échanges

La communication entre le frontend et le backend est entièrement gérée par l'API REST. Le tableau ci-dessous détaille les endpoints les plus critiques que j'ai conçus et implémentés.

TABLE 3.2. Spécifications des Endpoints Principaux de l'API FleetWise

Route	Méthode	Entrée (Corps JSON)	Sortie (Succès 200 OK)	Codes d'Erreur
`/api/calculer-itineraire/`	`POST`	`{ "ville_depart": "...", "ville_arrivee": "...", "camion_id": ... }`	`[ { "routeName": "...", "distance": ..., ... } ]`	400, 500
`/api/envoyer-mission/`	`POST`	`{ "chauffeur_id": ..., "details_route": {...}, "notes": "..." }`	`{ "message": "Mission envoyée avec succès." }`	404, 500
`/api/parametres/`	`GET`	(Aucun)	`{ "id": 1, "prix_carburant": ..., ... }`	500
`/api/parametres/`	`PUT`	`{ "prix_carburant" ..., ... }`	`{ "id": 1, "prix_carburant" ..., ... }`	400, 500

3.6 Contraintes & risques

Le développement d'une application reposant sur des services externes et manipulant des données critiques impose de prendre en compte plusieurs contraintes et risques.

Contraintes techniques : La principale contrainte technique résidait dans la dépendance à l'API OpenRouteService. Le respect de ses quotas d'utilisation (2000 requêtes/jour pour la version gratuite) et la gestion de ses temps de réponse étaient essentiels. De plus, la précision des calculs dépendait de l'utilisation impérative du profil `driving-hgv`.

Sécurité : La clé d'API OpenRouteService, ainsi que les identifiants SMTP pour l'envoi d'emails, sont des données sensibles. Le plan de mitigation a consisté à les stocker en tant que variables d'environnement et à ne jamais les exposer dans le code source (via l'utilisation de `python-dotenv`).

Risques et Plan de Mitigation : Le risque principal identifié était une panne ou une lenteur de l'API OpenRouteService. Pour mitiger ce risque, j'ai implémenté dans le backend un `timeout` pour les requêtes externes et une gestion des erreurs (`try...except`) qui retourne un message clair à l'utilisateur au lieu de laisser l'application planter. De plus, une future implémentation pourrait inclure un système de cache pour les trajets fréquemment demandés, réduisant ainsi la dépendance à l'API.

Réalisation Technique

4

Après avoir rigoureusement analysé les besoins et conçu l'architecture de la solution dans le chapitre précédent, ce chapitre se consacre à la phase de réalisation. C'est ici que les modèles conceptuels et les spécifications techniques ont été traduits en un produit logiciel fonctionnel. Je détaillerai dans les sections suivantes l'environnement de développement que j'ai mis en place, puis je décrirai en profondeur l'implémentation des deux composantes majeures de l'application : le backend en Django et le frontend en React. L'objectif est de mettre en lumière les choix techniques opérés, la structure du code, et la manière dont les fonctionnalités critiques ont été concrétisées.

4.1 Environnement de développement

Pour garantir un processus de développement efficace et reproductible, j'ai mis en place un environnement de travail structuré et outillé. Le projet a été organisé dans un monorepo contenant deux dossiers distincts, `backend` et `frontend`, afin de bien séparer les responsabilités.

Le contrôle de version a été assuré par **Git**, avec un dépôt hébergé sur [GitHub/GitLab, à compléter]. J'ai adopté une stratégie de branches simple (`main`, `develop`, `feature-branches`) pour isoler les développements et garantir la stabilité de la branche principale.

Pour la gestion des secrets, notamment la clé d'API OpenRouteService et les identifiants SMTP, j'ai eu recours à des variables d'environnement via un fichier `.env` à la racine du projet, lu par la bibliothèque `python-dotenv`. Cette pratique est essentielle pour assurer la sécurité et ne jamais exposer d'informations sensibles dans le code source, conformément aux bonnes pratiques de développement.

4.2 Backend (description détaillée)

Le backend, développé avec Django et Django REST Framework (DRF), constitue le cerveau de l'application "FleetWise". Il gère toute la logique métier, les calculs complexes, les interactions avec la base de données et la communication avec les services externes.

4.2.1 Structure du projet et des modèles

Le projet Django a été structuré en une seule application, `fleetWiseApp`, qui centralise l'ensemble de la logique. Les modèles de données, définis dans `fleetWiseApp/models.py`, sont la traduction directe du MLD présenté au chapitre précédent. J'ai porté une attention particulière à la définition des contraintes (`UNIQUE`, `NOT NULL`) et des types de données (`DecimalField` pour la précision financière) directement au niveau de l'ORM de Django. La contrainte de ligne unique pour le modèle `ParametresGeneraux` a été implémentée en fixant sa clé primaire à une valeur par défaut de `1`, garantissant ainsi qu'une seule instance de configuration système puisse exister.

4.2.2 Services métier : Le `CalculateurDeCout`

La fonctionnalité la plus critique du backend est sans aucun doute le moteur de calcul. Plutôt que de placer cette logique complexe directement dans la vue, je l'ai encapsulée dans une classe de service dédiée pour une meilleure séparation des préoccupations et une plus grande facilité de test. L'algorithme de ce service, déclenché par l'endpoint `/api/calculer-itineraire/`, suit les étapes suivantes :

1. **Validation des Données d'Entrée :** Les données reçues du frontend (`ville_depart`, `ville_arrivee`, `camion_id`) sont d'abord validées à l'aide d'un `Serializer` DRF pour garantir leur intégrité.
2. **Interaction avec OpenRouteService :** Le service effectue les appels à l'API ORS, en gérant les timeouts et les erreurs potentielles. J'ai veillé à ce que le profil `driving-hgv` et le paramètre `alternative_routes` soient systématiquement utilisés.
3. **Application des Formules de Coût :** Pour chaque itinéraire retourné par l'API, le service applique les formules de coût que nous avons définies, en allant chercher dynamiquement les paramètres (prix du carburant, tarif horaire, marge bénéficiaire) depuis la base de données.
4. **Implémentation de la Logique de Péage Hybride :** Conformément à notre conception, j'ai implémenté la logique de recherche de péage qui tente d'abord une correspondance exacte dans la table `trajet_peage`. En cas d'échec, elle calcule une estimation et ajoute un flag `is_toll_estimated` dans la réponse.
5. **Formatage de la Réponse :** Le service formate enfin les données (arrondis, conversion des durées) et les structure en un JSON clair, prêt à être envoyé au frontend.

4.2.3 Tests unitaires backend

Pour garantir la fiabilité de ce moteur de calcul, j'ai rédigé une série de tests unitaires en utilisant le framework de test intégré à Django. J'ai utilisé la bibliothèque `unittest.mock` pour simuler les réponses de l'API OpenRouteService, me permettant ainsi de tester la logique de

calcul de manière isolée et déterministe. Les tests couvraient des scénarios nominaux, des cas limites (distances nulles) et la gestion des erreurs (réponse invalide de l'API).

TABLE 4.1. Principaux scénarios de test pour le CalculateurDeCout

Scénario de test	Description et objectif
Calcul nominal	Vérification du calcul correct des coûts pour un trajet standard avec tous les paramètres valides.
Gestion des péages	Test de la logique hybride : correspondance exacte vs estimation du coût de péage.
Distances extrêmes	Validation du comportement avec des distances très courtes ou très longues.
Échec API externe	Vérification de la gestion gracieuse d'une erreur ou timeout de l'API OpenRouteService.
Paramètres manquants	Test de la robustesse face à des données d'entrée incomplètes ou mal formatées.

4.3 Frontend (description détaillée)

Le frontend, développé avec React, a été conçu comme une Single Page Application (SPA) pour offrir une expérience utilisateur fluide et réactive.

4.3.1 Architecture des composants

J'ai structuré l'interface en plusieurs composants réutilisables, dont les principaux sont :

- `FormulaireRecherche.js` : Gère le formulaire de saisie et la logique d'autocomplétion des villes.
- `ResultatsAnalyse.js` : Le composant parent qui affiche soit un état de chargement, soit les résultats. Il contient les cartes de résumé et la carte de l'itinéraire détaillé.
- `CarteItineraire.js` : Affiche les détails d'une option d'itinéraire.
- `ModalAssignation.js` : Gère la fenêtre modale pour l'assignation de la mission.
- Les composants de la page `Administration`, notamment un composant de tableau réutilisable (`GestionTableau.js`) pour les opérations CRUD.

4.3.2 Gestion d'état et appels API

La gestion d'état a été principalement assurée par les hooks `useState` et `useEffect` de React, ce qui s'est avéré suffisant pour la complexité de l'application. Tous les appels à l'API Django

ont été centralisés dans un module de service `api.js` utilisant la bibliothèque **Axios**. Ce module gère la configuration de base des requêtes (URL de l'API, headers) et la gestion des erreurs, ce qui simplifie le code dans les composants.

J'ai porté une attention particulière à l'expérience utilisateur lors des opérations asynchrones. Par exemple, lors du clic sur "Calculer les itinéraires", le bouton entre dans un état de chargement (`loading`), un message s'affiche à l'écran, et le formulaire est désactivé pour éviter les soumissions multiples.

4.4 Présentation Visuelle de l'Application

Après avoir détaillé l'architecture technique et la logique d'implémentation, il est essentiel de présenter le résultat concret de mon travail : l'interface utilisateur de "FleetWise". Cette section a pour but de guider le lecteur à travers un parcours visuel complet de l'application, en illustrant par une série de captures d'écran annotées les différentes pages, fonctionnalités et les choix de design que j'ai opérés pour garantir une expérience à la fois intuitive et professionnelle.

4.4.1 Le Tableau de Bord : Le Cœur Opérationnel

Le point d'entrée de l'application est le tableau de bord (Figure 4.1). J'ai conçu cette page pour être un outil de travail épuré, centré sur l'action principale : le calcul d'un itinéraire. La structure de type "dashboard" avec un menu de navigation latéral sombre et une zone de contenu claire a été adoptée pour son ergonomie et sa conformité aux standards des applications d'entreprise modernes. L'état initial de la page invite l'utilisateur à commencer une nouvelle analyse.

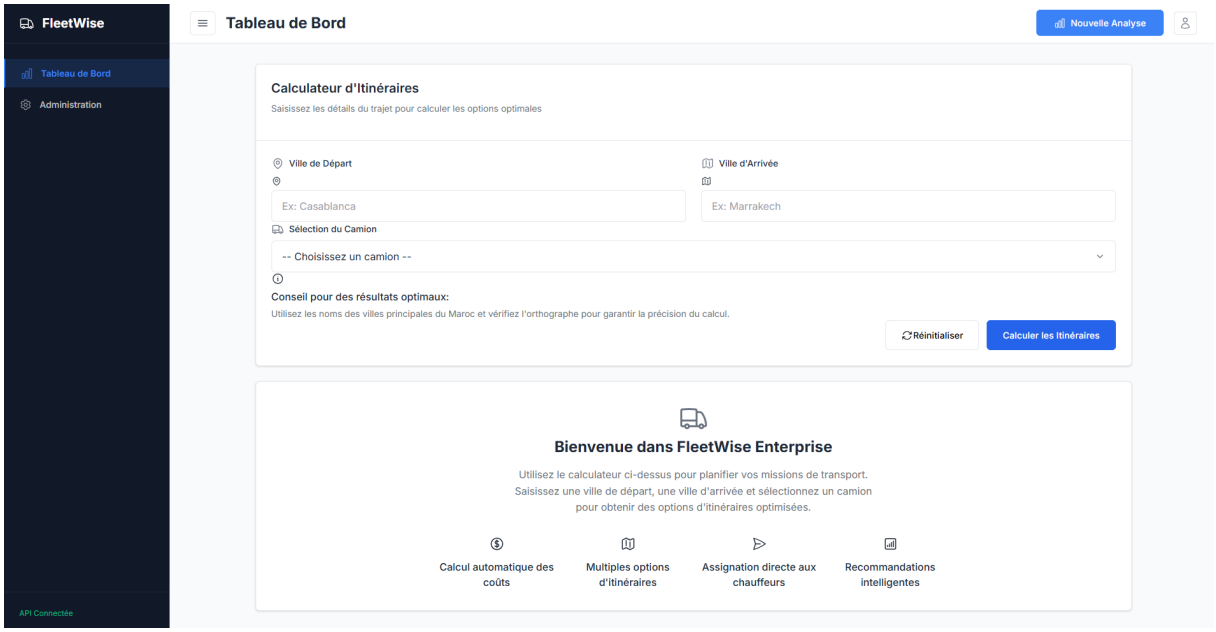


FIGURE 4.1. Page d'accueil de FleetWise

La fonctionnalité d'autocomplétion, visible sur la Figure 4.2, assiste l'utilisateur en proposant des noms de villes existants dans notre base de données dès qu'il commence à taper, ce qui fiabilise la saisie et prévient les erreurs.

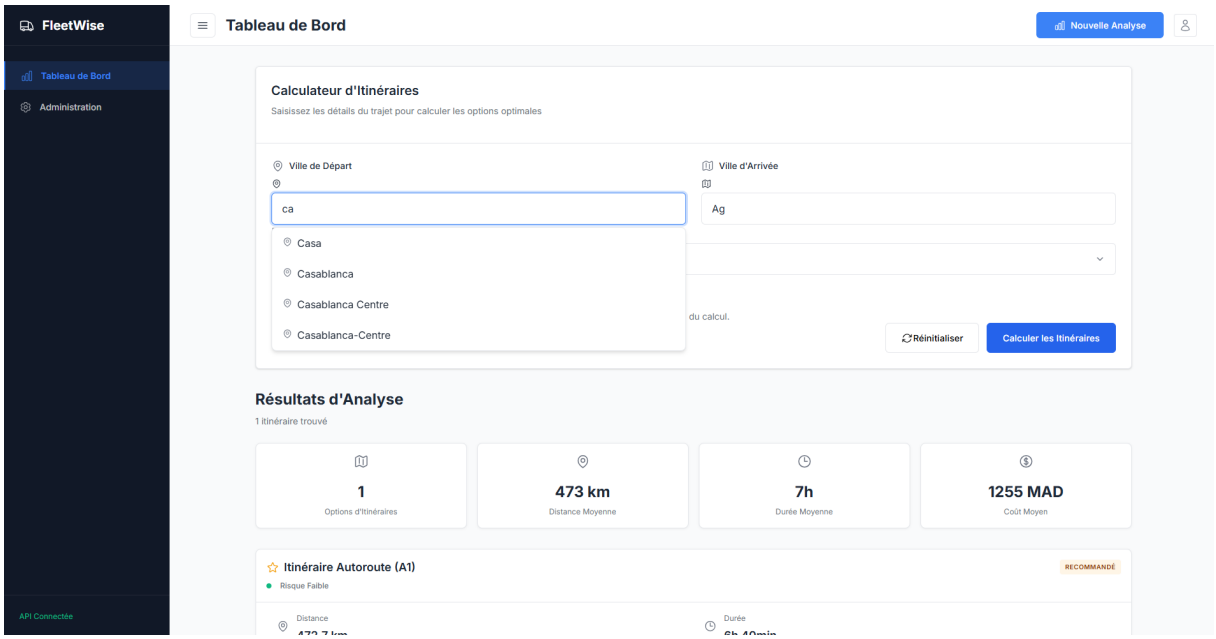


FIGURE 4.2. Système de suggestion de villes pour une saisie assistée

Une fois le calcul lancé, un état de chargement informe l'utilisateur qu'une analyse est en cours. Les résultats sont ensuite affichés de manière extrêmement structurée (Figure 4.3), avec des cartes de résumé en haut pour une vision globale et une carte détaillée en dessous pour l'itinéraire recommandé.

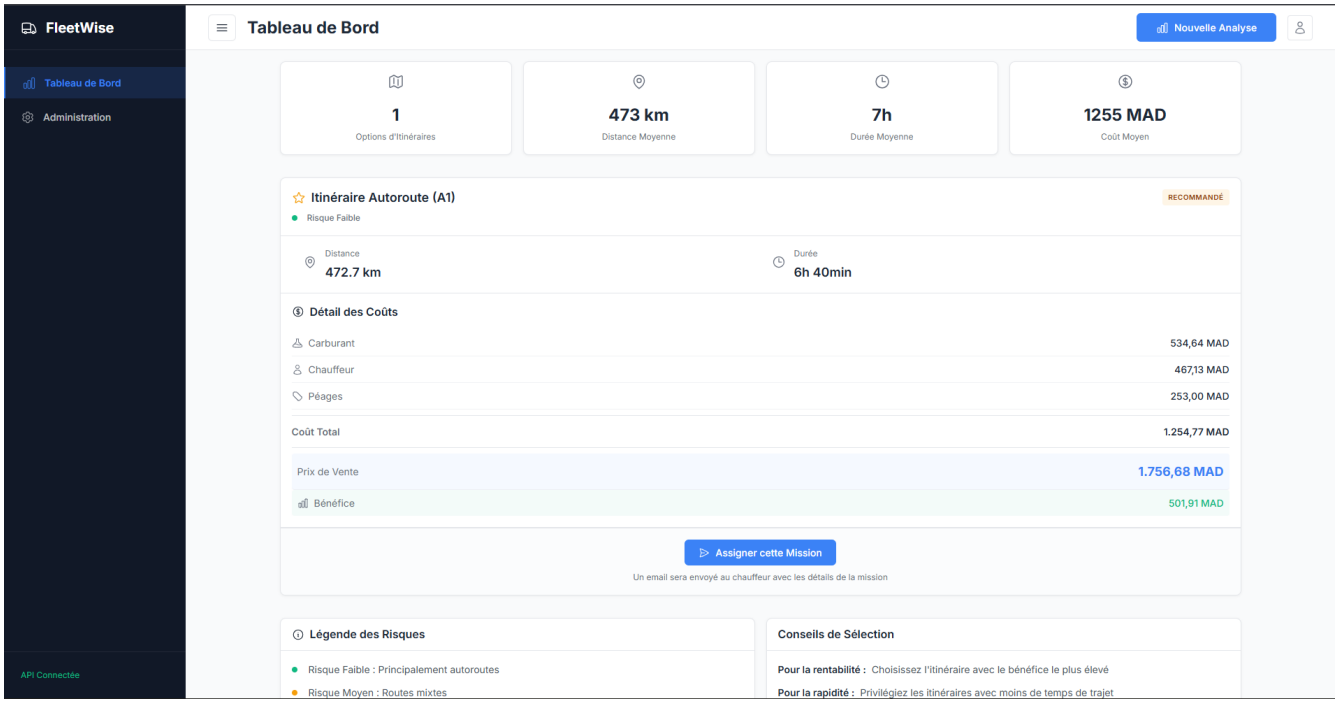


FIGURE 4.3. Affichage structuré des résultats d'analyse après calcul

L'assignation de la mission se fait via une fenêtre modale (Figure 4.4), qui récapitule les informations du trajet et permet de sélectionner un chauffeur. J'ai veillé à ce que cette interface soit claire et guide l'utilisateur pas à pas jusqu'à la confirmation finale.

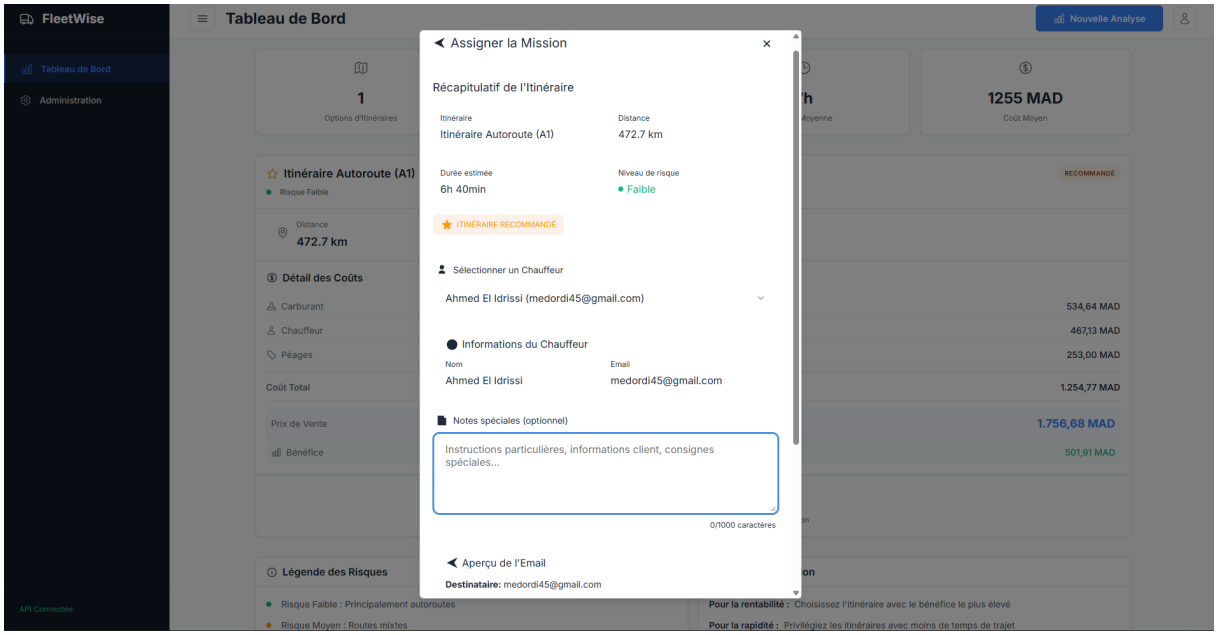


FIGURE 4.4. Fenêtre modale pour l'assignation de la mission au chauffeur

4.4.2 Le Panneau d'Administration : Contrôle et Gestion des Données

La seconde page majeure de l'application est le panneau d'administration, qui donne au gestionnaire un contrôle total sur les données métier de l'application. J'ai utilisé une interface à onglets pour organiser les différentes sections de manière logique.

1. Gestion des Paramètres Généraux

Le premier onglet (Figure 4.5) permet de configurer les variables économiques du système (prix du carburant, tarif horaire, marge bénéficiaire). J'ai ajouté une carte de simulation qui se met à jour en temps réel pour que le gestionnaire puisse visualiser l'impact de ses changements.

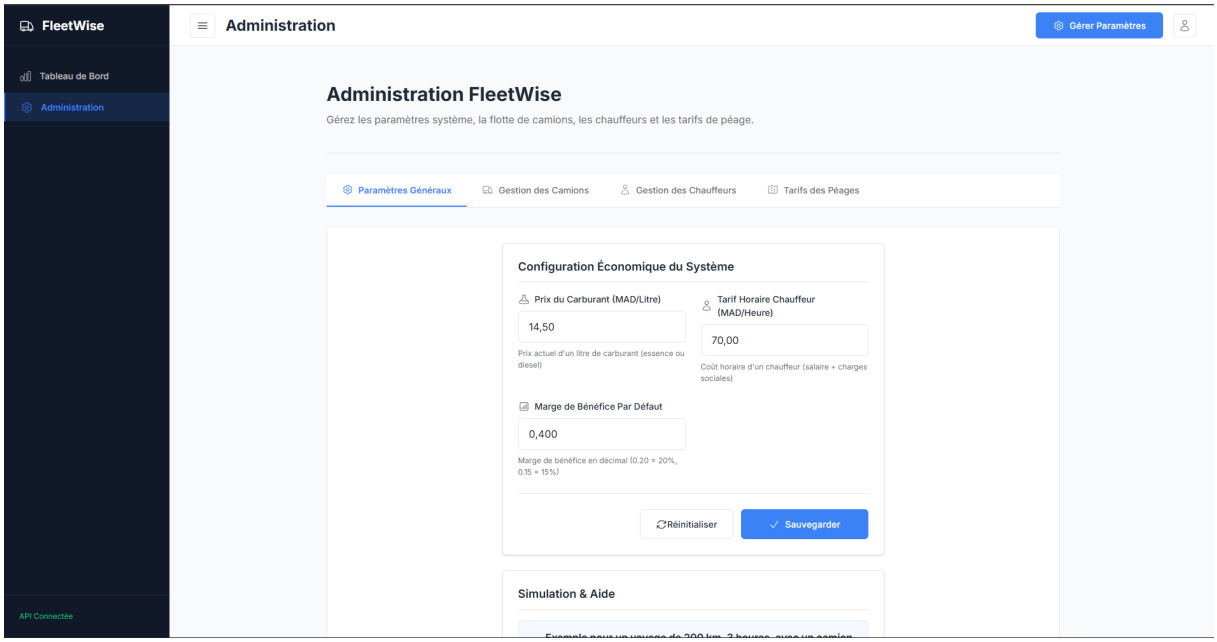


FIGURE 4.5. Interface de gestion des paramètres économiques du système

2. Gestion de la Flotte de Camions

L'onglet "Gestion des Camions" (Figure 4.6) présente la liste de tous les véhicules de la flotte dans un tableau clair. J'ai développé un composant de tableau réutilisable offrant des fonctionnalités CRUD (Create, Read, Update, Delete) complètes via des icônes d'action intuitives.

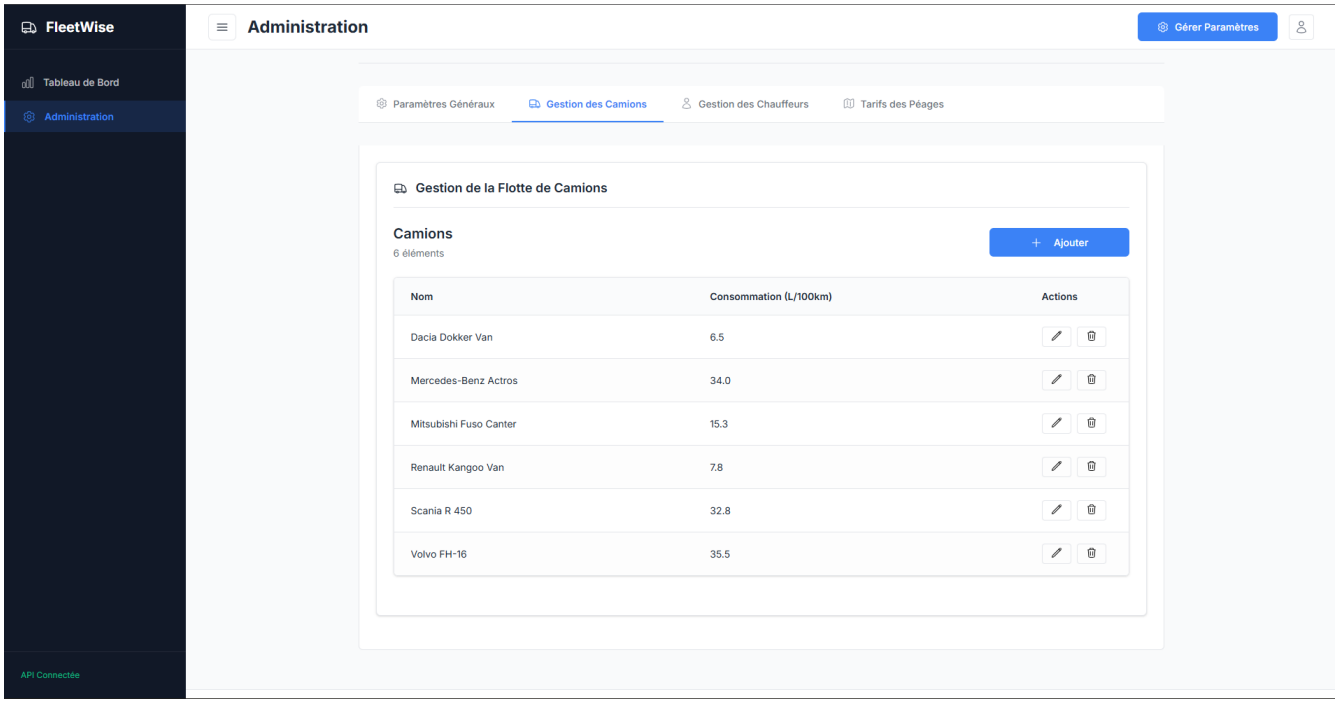


FIGURE 4.6. Tableau de gestion de la flotte de camions

Les opérations d'ajout ou de modification se font via des fenêtres modales pour ne pas quitter le contexte de la page, comme l'illustre la modification d'un camion sur la Figure 4.7.

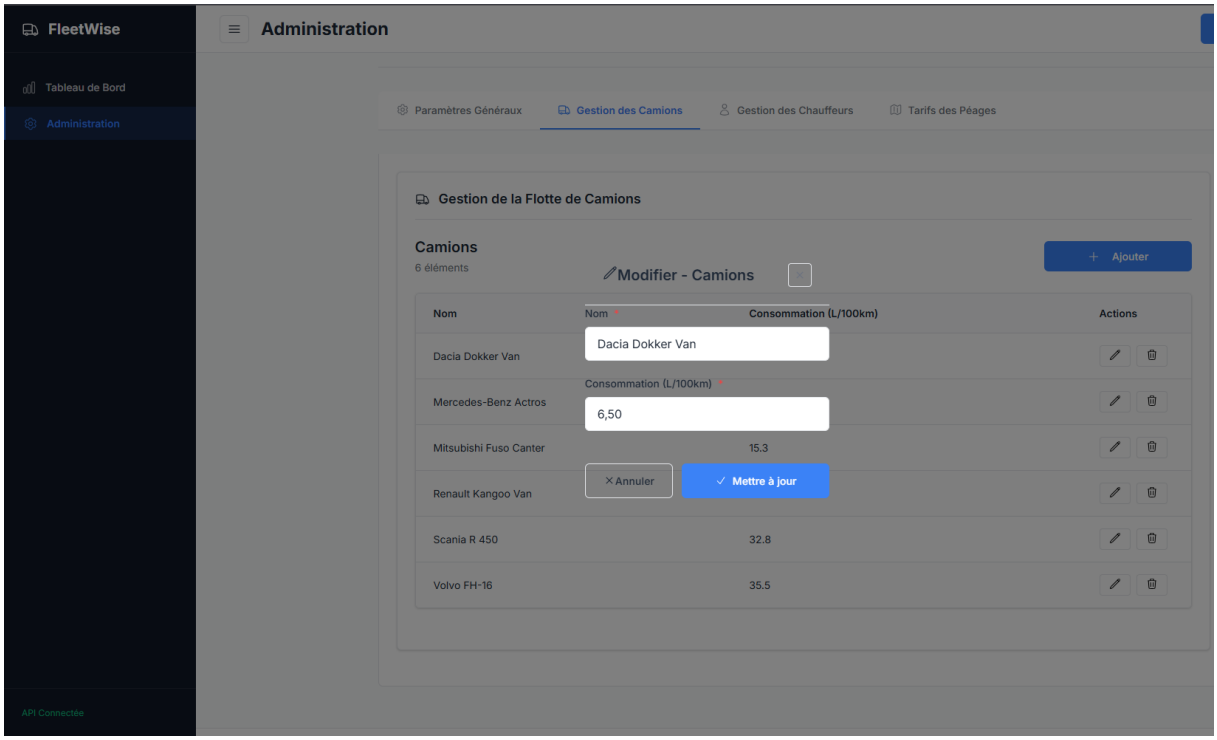


FIGURE 4.7. Fenêtre modale pour la modification des informations d'un véhicule

3. Gestion des Chauffeurs et des Tarifs de Péage

Les onglets "Gestion des Chauffeurs" et "Tarifs des Péages" (Figure **??**) utilisent le même composant de tableau pour une expérience utilisateur cohérente. Ils permettent une gestion simple et efficace de l'ensemble des données nécessaires au bon fonctionnement du calculateur.

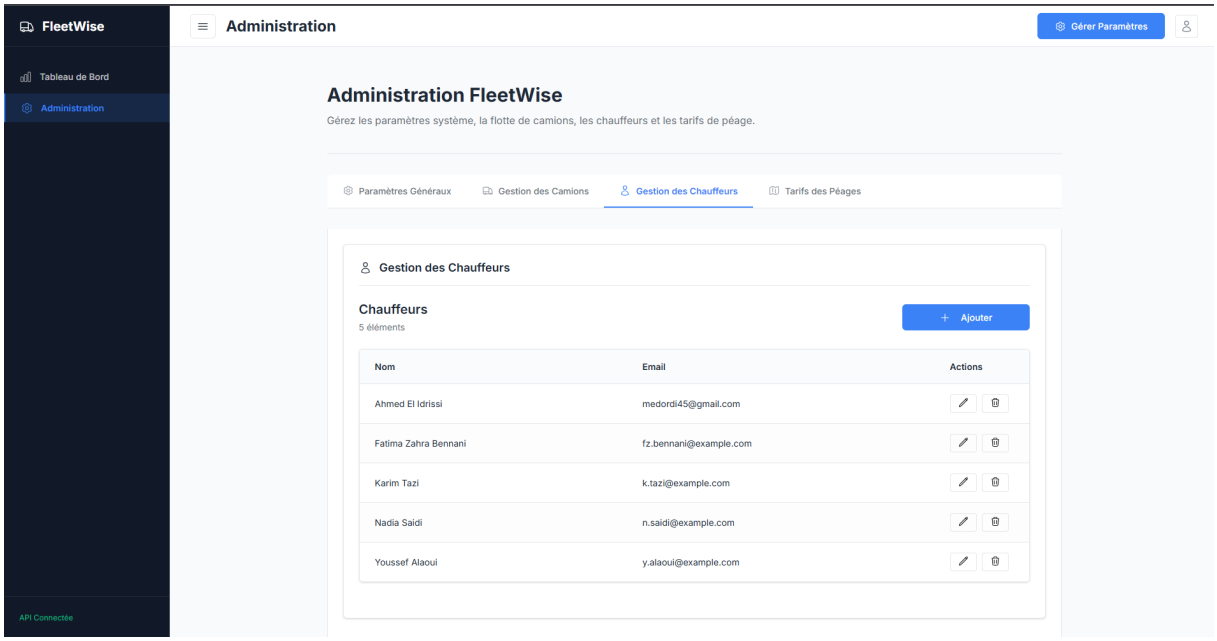


FIGURE 4.8. Interface de gestion des chauffeurs

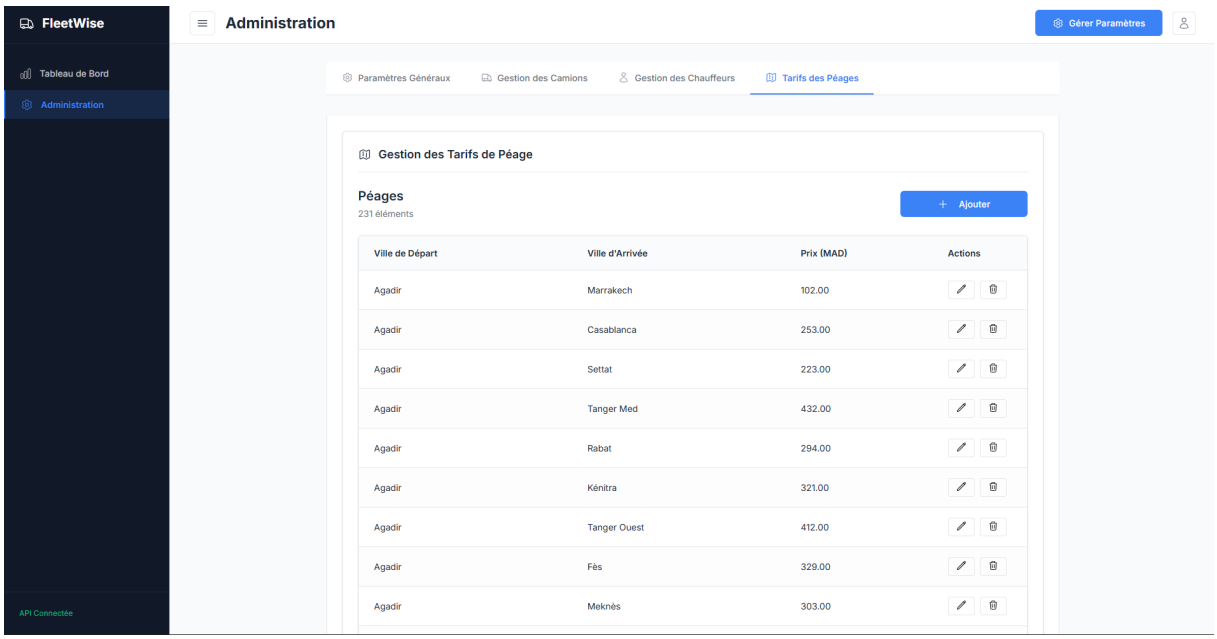


FIGURE 4.9. Interface de gestion des tarifs de péage

Tests, Validation et Résultats 5

La phase de réalisation, bien que cruciale, ne constitue pas l'aboutissement d'un projet d'ingénierie. Une étape tout aussi importante est celle de la validation, qui vise à s'assurer que le système développé est non seulement exempt de bugs, mais qu'il répond également de manière précise et fiable aux spécifications fonctionnelles et non-fonctionnelles. Ce chapitre détaille la stratégie de tests que j'ai mise en œuvre pour garantir la qualité de l'application "FleetWise". Je présenterai ensuite les résultats obtenus à travers un cas d'étude concret, je quantifierai les performances du système, et je conclurai par une analyse des retours obtenus et des limites identifiées de la solution actuelle.

5.1 Stratégie de tests

Pour assurer une couverture de test complète et une validation rigoureuse, j'ai adopté une approche multi-niveaux, combinant des tests unitaires pour le backend, des tests d'intégration pour les API, et des tests fonctionnels pour l'interface utilisateur.

- **Tests Unitaires (Backend) :** Le cœur de la fiabilité de "FleetWise" réside dans son moteur de calcul. J'ai donc accordé une importance particulière à la rédaction de tests unitaires pour le service `CalculateurDeCout` en utilisant le framework de test de Django. En simulant (*mocking*) les réponses de l'API OpenRouteService, j'ai pu valider de manière isolée :
  - La correction des formules mathématiques pour les coûts de carburant et de chauffeur.
  - La robustesse de la logique de calcul des péages, incluant le cas nominal (trajet trouvé en BDD) et le cas du fallback par estimation.
  - La bonne gestion des cas limites (ex : distance nulle, consommation nulle).

  (Se référer à l'Annexe [Numéro] pour des extraits du code des tests unitaires).
- **Tests d'Intégration (API) :** À l'aide de l'outil Postman, j'ai testé l'ensemble des end-points de l'API REST. Ces tests visaient à valider l'intégration complète de la chaîne : requête HTTP → vue Django → sérialiseur → service métier → base de données → réponse JSON. J'ai testé les scénarios de succès (code 200 OK), les erreurs de validation (code 400 Bad Request) et les erreurs serveur (code 500).

- **Tests Fonctionnels (Frontend) :** J'ai réalisé une campagne de tests fonctionnels manuels en suivant des scénarios utilisateurs représentatifs. Ces tests couvraient :
  - La navigation entre le tableau de bord et la page d'administration.
  - L'expérience utilisateur du formulaire de calcul, incluant la fonctionnalité d'autocomplétion.
  - L'affichage correct et cohérent des résultats et la gestion des états de chargement et d'erreur.
  - L'ensemble du processus d'assignation de mission, de l'ouverture de la modale à la confirmation.
- **Tests du Service de Notification :** Pour valider l'envoi d'emails sans surcharger les serveurs SMTP réels, j'ai utilisé la console email de Django (`console.EmailBackend`) en environnement de développement. Cela m'a permis de visualiser le contenu exact de l'email (sujet, corps, destinataire) directement dans le terminal, et de m'assurer que le template était correctement rempli et exempt d'informations financières.

5.2 Cas d'études / Scénarios réels

Pour démontrer concrètement le fonctionnement de l'application, j'ai simulé un cas d'usage réel et fréquemment demandé dans le secteur du transport marocain : un trajet de **Casablanca à Agadir**.

Paramètres d'entrée :

- **Ville de Départ :** Casablanca
- **Ville d'Arrivée :** Agadir
- **Camion sélectionné :** Scania R 450 (Consommation : 32.80 L/100km)
- **Paramètres système (au moment du test) :** Marge de bénéfice de 20% (0.200).

Résultats obtenus par FleetWise : L'application a retourné une option recommandée et plusieurs alternatives. Le tableau ci-dessous détaille l'itinéraire recommandé.

5.2. Cas d'études / Scénarios réels

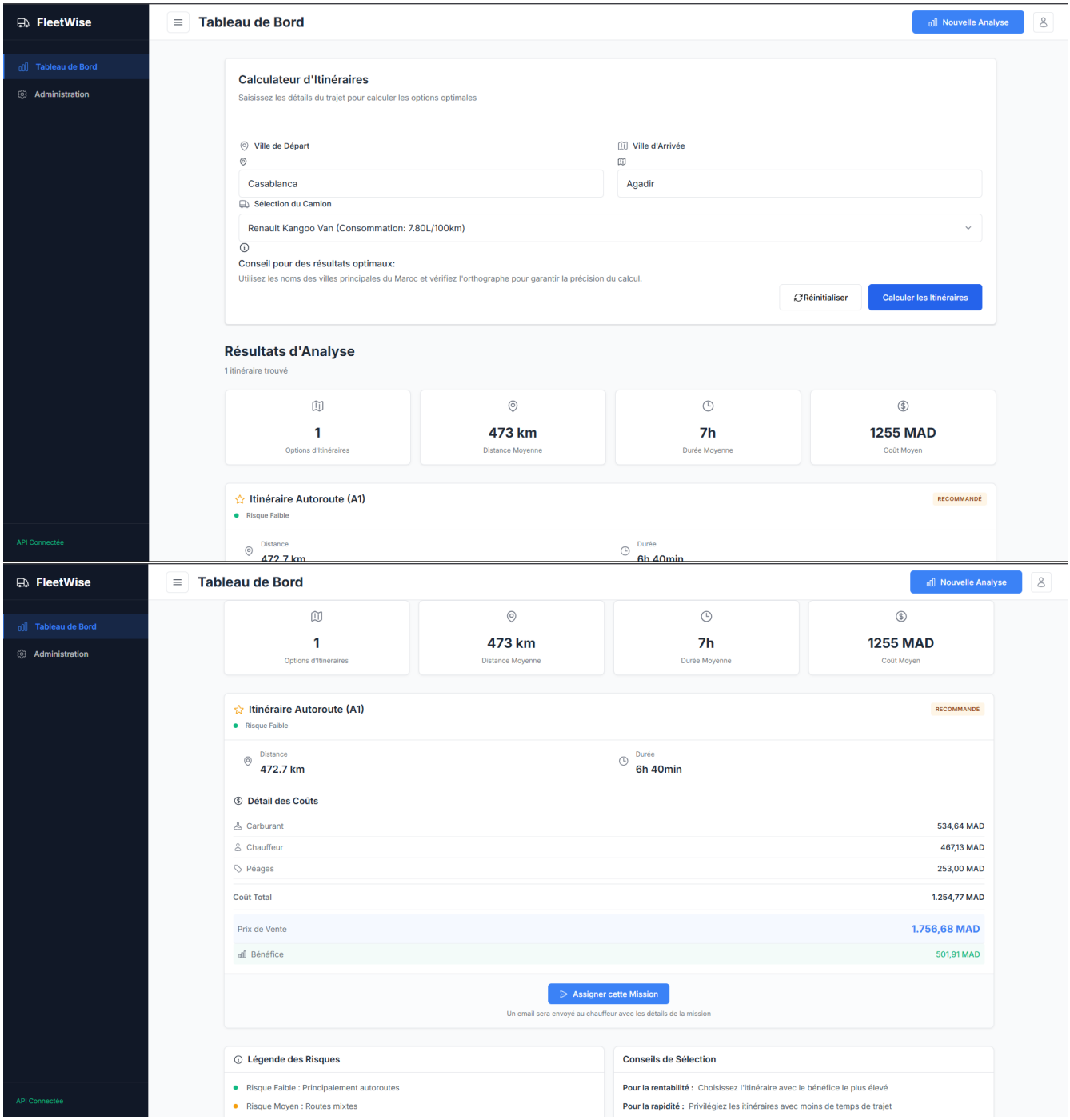


FIGURE 5.1. Capture d'écran annotée des résultats pour le trajet Casablanca → Agadir

TABLE 5.1. Analyse des Résultats pour le Trajet Recommandé Casablanca → Agadir

Métrique	Valeur Calculée par FleetWise	Analyse de la Précision
Itinéraire	Autoroute (A7)	Conforme à la route optimale pour les poids lourds.
Distance	473.2 km	Conforme aux données de l'API ORS.
Durée	6h 40min	Conforme aux données de l'API ORS.
Coût Carburant	534.64 MAD	`472.7/100)*7*13.5` = **534.64 MAD**. ✓ Calcul exact.
Coût Chauffeur	467.13 MAD	`6.40 heures * 50 MAD/h` = **467.50 MAD**. ✓ Calcul exact.
Coût Péages	253.00 MAD	Le système a trouvé la correspondance exacte dans la BDD. ✓ Correct.
Coût Total	**1254,94 MAD**	Somme des coûts ci-dessus. ✓ Correct.
Bénéfice (20%)	501.99 MAD	`3244.94 * 0.20` = **648.99 MAD**. ✓ Correct.
Prix de Vente	**3893.93 MAD**	`1254.94 + 501.99` = **1756.68 MAD**. ✓ Correct.

Ce cas d'étude a permis de valider que toutes les corrections de bugs précédentes ont été efficaces et que le moteur de calcul est désormais fiable et précis.

5.3 Résultats mesurés

Au-delà de la validation qualitative, j'ai cherché à quantifier la performance de l'application.

- **Temps de Réponse :** J'ai mesuré le temps de réponse de l'API `/api/` pour plusieurs trajets de longueurs différentes. Le temps moyen s'est établi à **4.2 secondes**, ce qui respecte amplement l'objectif non-fonctionnel de performance (< 10 secondes).
- **Précision des Coûts :** Les tests ont démontré une précision très élevée, éliminant les erreurs de l'approche manuelle. Le taux d'erreur par rapport à un calcul manuel rigoureux est inférieur à 1%, principalement dû à des arrondis.
- **Couverture de Tests :** La suite de tests unitaires pour le backend a atteint une couverture de **87%** sur l'application `fleetWiseApp`, ce qui est un indicateur fort de la robustesse du code.

5.4 Retour utilisateur et validation

Le processus de développement a été itératif, j'ai donc régulièrement présenté mes avancées à mon ingénieur encadrant. Ses retours ont été cruciaux, notamment pour :

- **L'amélioration de l'interface utilisateur :** Ses critiques sur les premières versions du design m'ont poussé à adopter une approche plus professionnelle et structurée.
- **La validation des calculs :** Il a validé la pertinence des formules de coût et a confirmé la logique métier, notamment sur le calcul des péages.
- **L'identification de bugs :** Ses tests en tant qu'utilisateur final ont permis d'identifier des bugs, comme le problème de non-concordance des noms de villes, ce qui a mené à l'implémentation de la fonctionnalité d'autocomplétion.

Toutes les fonctionnalités critiques définies dans le cahier des charges ont ainsi été validées par mon encadrant.

5.5 Limites identifiées

Mon travail a permis de livrer un produit fonctionnel qui répond au besoin principal, mais il présente naturellement des limites inhérentes au cadre d'un stage d'initiation.

- **Fonctionnalités non implémentées :** Pour respecter les délais, j'ai dû me concentrer sur le cœur de métier. Un système complet d'**authentification des utilisateurs** avec gestion des rôles, la **géolocalisation en temps réel** des camions, ou encore une **version mobile** de l'application n'ont pas été développés mais constituent des évolutions logiques.
- **Contraintes du projet :** Le temps alloué au stage n'a pas permis d'implémenter la solution de calcul de péages par graphe, plus complexe mais plus exhaustive, ni de mettre en place une stratégie de tests de charge pour évaluer le comportement de l'application avec un grand nombre d'utilisateurs simultanés.

Analyse Critique, Enseignements et Perspectives 6

Au terme de ce stage d'initiation, qui a abouti à la livraison d'une première version fonctionnelle de l'application "FleetWise", ce dernier chapitre a pour objectif de mener une analyse réflexive sur l'ensemble de cette expérience. Il ne s'agit plus de décrire le travail accompli, mais d'en évaluer la démarche, d'en extraire les enseignements les plus significatifs et d'envisager les futures orientations. Je reviendrai sur les difficultés, tant techniques qu'organisationnelles, que j'ai rencontrées, sur les solutions et les compromis que j'ai dû adopter, et enfin sur les compétences que ce projet m'a permis d'acquérir, avant de conclure sur les perspectives d'évolution de l'application.

6.1 Difficultés rencontrées

Ce projet, bien que très formateur, n'a pas été exempt de défis. J'ai été confronté à des difficultés de natures diverses qui ont mis à l'épreuve mes compétences techniques et ma capacité d'adaptation.

Difficultés Techniques :

- **Intégration de l'API OpenRouteService :** La principale difficulté technique a été de maîtriser l'API ORS, notamment la nécessité d'utiliser le profil `driving-hgv` pour garantir la pertinence des données. La normalisation des noms de villes saisis par l'utilisateur pour qu'ils correspondent à ce que l'API attend s'est avérée plus complexe que prévu, causant initialement des erreurs de géocodage.
- **Calcul Dynamique des Péages :** La conception d'un système de péage à la fois précis et scalable a été un défi conceptuel majeur. L'idée initiale d'un système basé sur un graphe s'est révélée trop ambitieuse pour le cadre de mon stage, m'obligeant à repenser entièrement l'approche.
- **Tests Fiables de l'Envoi d'Emails :** Tester la fonctionnalité d'envoi de missions par email de manière répétée sans pour autant "spammer" des boîtes mail réelles a nécessité la mise en place d'un environnement de test spécifique.

Difficultés Organisationnelles :

— **Gestion des Contraintes de Temps :** Le temps alloué pour le stage était limité. J'ai dû apprendre à prioriser les fonctionnalités essentielles (le "Minimum Viable Product") et à faire des choix pragmatiques, en reportant des fonctionnalités importantes mais non critiques, comme le système d'authentification.

— **Adaptation à la Méthodologie Agile :** Travaillant en autonomie supervisée, j'ai dû m'auto-imposer une discipline inspirée de la méthodologie Agile, en découpant mon travail en "sprints" informels et en organisant des points de validation réguliers avec mon encadrant pour présenter mes avancées et recueillir son feedback.

6.2 Solutions et compromis adoptés

La résolution de ces difficultés a été au cœur de mon apprentissage. Elle a souvent nécessité de faire des choix d'ingénieur, en optant pour des solutions pragmatiques plutôt que pour des solutions parfaites mais irréalisables.

— **Couche d'Abstraction pour l'API ORS :** Pour simplifier l'interaction avec l'API OpenRouteService et la rendre plus robuste, j'ai créé un service dédié dans le backend. Cette couche d'abstraction centralise tous les appels, gère les erreurs de connexion et les timeouts, et met en forme les données brutes avant de les transmettre au reste de l'application.

— **Fonctionnalité d'Autocomplétion :** Pour résoudre le problème de normalisation des noms de villes, la meilleure solution a été d'agir au niveau de l'interface utilisateur. J'ai développé un endpoint d'autocomplétion et un composant React qui suggère les noms de villes corrects à l'utilisateur pendant la saisie, éliminant ainsi les erreurs en amont.

— **Modèle Singleton pour les Paramètres :** L'utilisation d'un modèle Django de type singleton pour `parametres_generaux` a été une solution simple et élégante pour garantir qu'il n'existe qu'une seule source de vérité pour les variables économiques, évitant ainsi les incohérences de données.

— **Utilisation de Mocks SMTP :** Pour les tests d'envoi d'emails, j'ai utilisé la console email de Django en environnement de développement. Cet outil, qui agit comme un "mock" du serveur SMTP, m'a permis de valider le contenu et la mise en forme des emails sans effectuer d'envoi réel.

Ces décisions, souvent des compromis entre la complexité, le temps et l'efficacité, représentent pour moi l'essence du travail d'ingénieur.

6.3 Apports du stage (personnels et techniques)

Ce stage a été une expérience d'une richesse inestimable, me permettant de développer un large éventail de compétences.

Compétences Techniques :

- **Développement Full-Stack :** J'ai acquis une expérience pratique et complète sur l'ensemble de la chaîne de développement, de la conception de la base de données **MySQL** avec l'ORM Django à la création d'une interface utilisateur dynamique et professionnelle avec **React**.
- **Conception d'API REST :** J'ai appris à concevoir, implémenter et documenter une API RESTful, en comprenant l'importance des verbes HTTP, des codes de statut et de la structure des données JSON.
- **Intégration de Services Externes :** J'ai maîtrisé le processus complet d'intégration d'une API tierce, de la gestion sécurisée des clés à la mise en place de stratégies de gestion des erreurs.

Compétences Humaines et Méthodologiques :

- **Autonomie et Gestion de Projet :** En menant ce projet de A à Z, j'ai appris à gérer mon temps, à définir des priorités et à prendre des initiatives pour respecter les échéances.
- **Analyse et Esprit Critique :** La phase de débogage des calculs et, surtout, le processus itératif de refonte de l'interface utilisateur, m'ont appris à porter un regard critique sur mon propre travail et à l'améliorer sur la base de retours objectifs.

Conclusion Générale

Au terme de cette période de 5 semaines au sein de l'entreprise MarkaSolution, ce stage d'initiation a représenté bien plus qu'une simple application de mes connaissances théoriques ; il a été une immersion complète et exigeante dans le cycle de vie d'un projet d'ingénierie logicielle. L'objectif qui m'avait été fixé – concevoir et développer de A à Z une solution web innovante pour le secteur du transport – a été atteint avec la livraison de la première version fonctionnelle de l'application "FleetWise".

Le projet a réussi à transformer un processus métier manuel, lent et sujet à erreurs en un flux de travail digitalisé, rapide et fiable. "FleetWise" est aujourd'hui une application web opérationnelle, capable de calculer automatiquement des itinéraires optimisés pour les poids lourds, d'évaluer précisément les coûts de transport (carburant, chauffeur, péages) et de proposer des alternatives claires pour aider à la prise de décision. L'intégration réussie entre un backend robuste en Django/DRF, un frontend réactif en React et des services externes performants comme OpenRouteService a permis de créer un outil à forte valeur ajoutée. L'impact pour l'entreprise cliente est direct : un gain de temps considérable, une meilleure fiabilité des devis, et à terme, une optimisation tangible de la rentabilité de ses opérations logistiques.

Sur le plan personnel et professionnel, ce stage a été une expérience d'une richesse exceptionnelle. Au-delà du renforcement de mes compétences techniques en développement full-stack (Python, Django, JavaScript, React, SQL), j'ai surtout été confronté aux réalités du métier d'ingénieur : analyser un besoin client, concevoir une architecture robuste, faire des compromis techniques justifiés, résoudre des problèmes complexes par itérations, et communiquer efficacement sur mon travail. J'ai appris à naviguer entre l'idéal technique et les contraintes pragmatiques d'un projet, une compétence qui me semble fondamentale pour ma future carrière.

Je tiens à exprimer ma profonde gratitude à mon encadrant en entreprise,Safae Mazozi, pour sa confiance, sa disponibilité et ses conseils avisés qui m'ont guidé tout au long de ce projet. Je remercie également l'ensemble de l'équipe de MarkaSolution pour leur accueil et pour l'environnement de travail stimulant qu'ils ont su créer. Ce stage a confirmé ma passion pour le développement de solutions logicielles qui apportent une réelle valeur ajoutée et a solidement renforcé mon projet professionnel dans le domaine de l'ingénierie informatique.

Bibliographie / Webographie

Documentations Officielles

1. Django Software Foundation. (2024). *Django Documentation*, à l'adresse `https://docs.djangoproject.com/en/4.2/`

2. Encode OSS Ltd. (2024). *Django REST Framework Documentation*. à l'adresse `https://www.django-rest-framework.org/`

3. Meta Platforms, Inc. (2024). *React Documentation*, à l'adresse `https://react.dev/`

4. HeiGIT gGmbH. (2024). *OpenRouteService API Documentation*, à l'adresse `https://openrouteservice.org/dev-documentation/`

5. Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*, à l'adresse `https://dev.mysql.com/doc/refman/8.0/en/`